Lecture 2.1

Course Name: Digital Image Processing
Course code: 4245

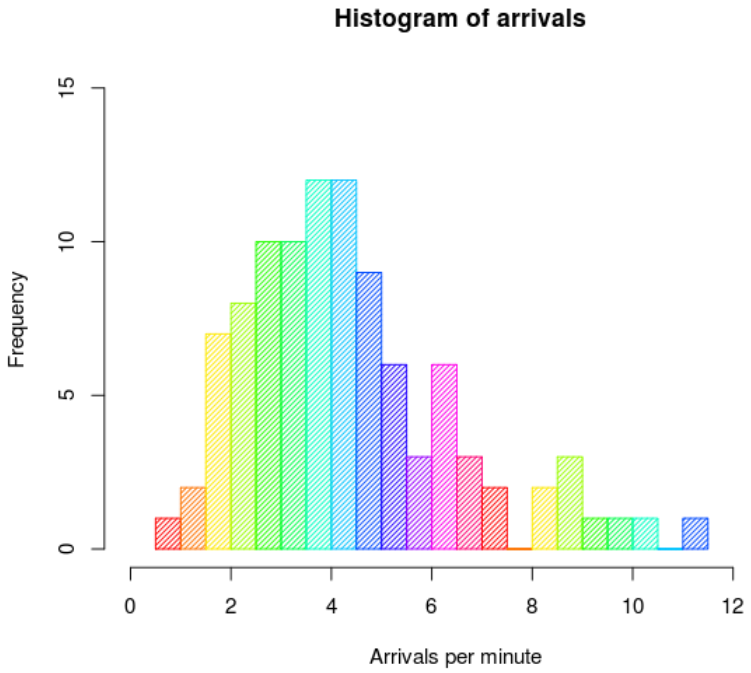
Histogram Processing and Equalization

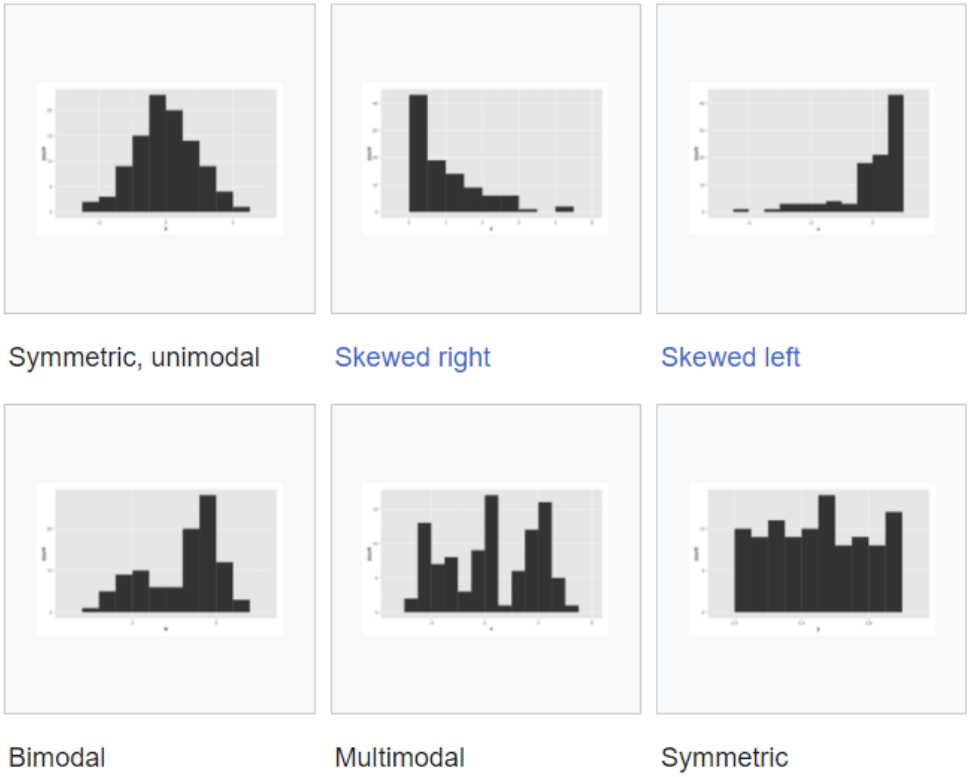
Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Histogram (Karl Pearson)

- A **histogram** is an approximate representation of the distribution of numerical data.

1. Whether an image is dark or light, and high or low contrast.





Histogram (Karl Pearson)

Brightness: Brightness is the intensity of light emit by a particular light source.

Contrast:

Maximum pixel intensity−Minimum pixel intensity

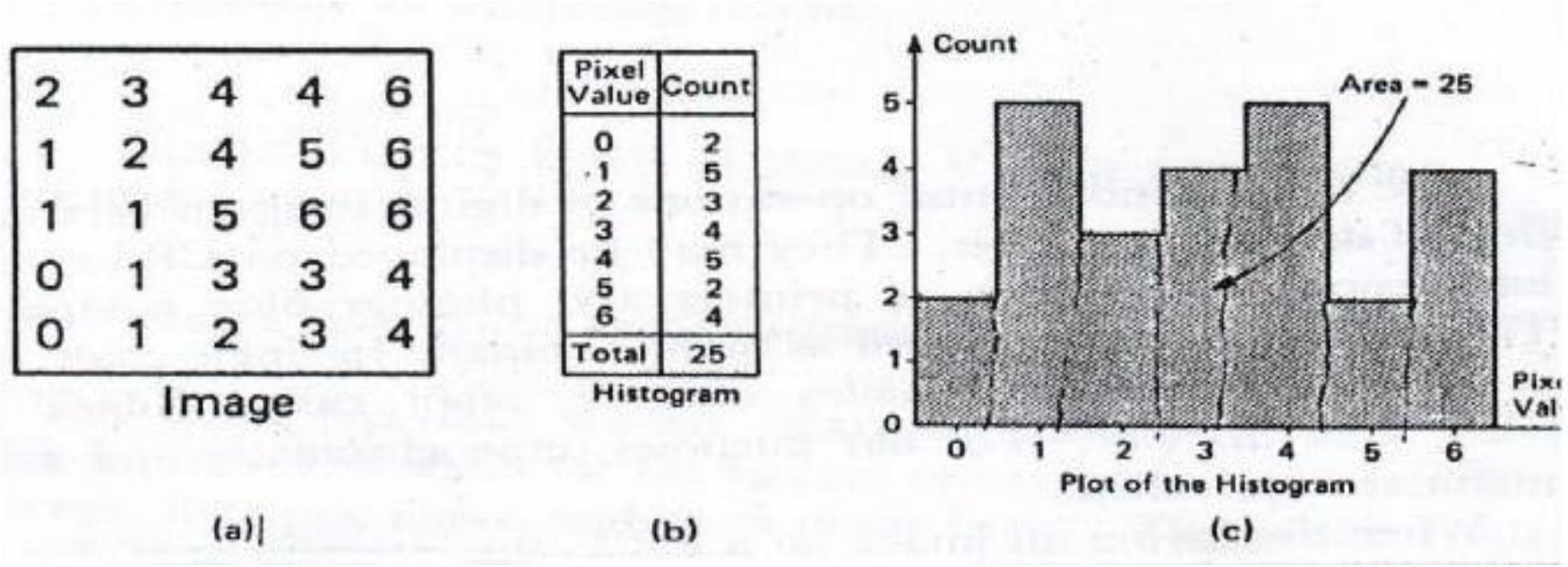
For example:

1. Very narrow histogram implies a **low-contrast image**

2. Histogram skewed

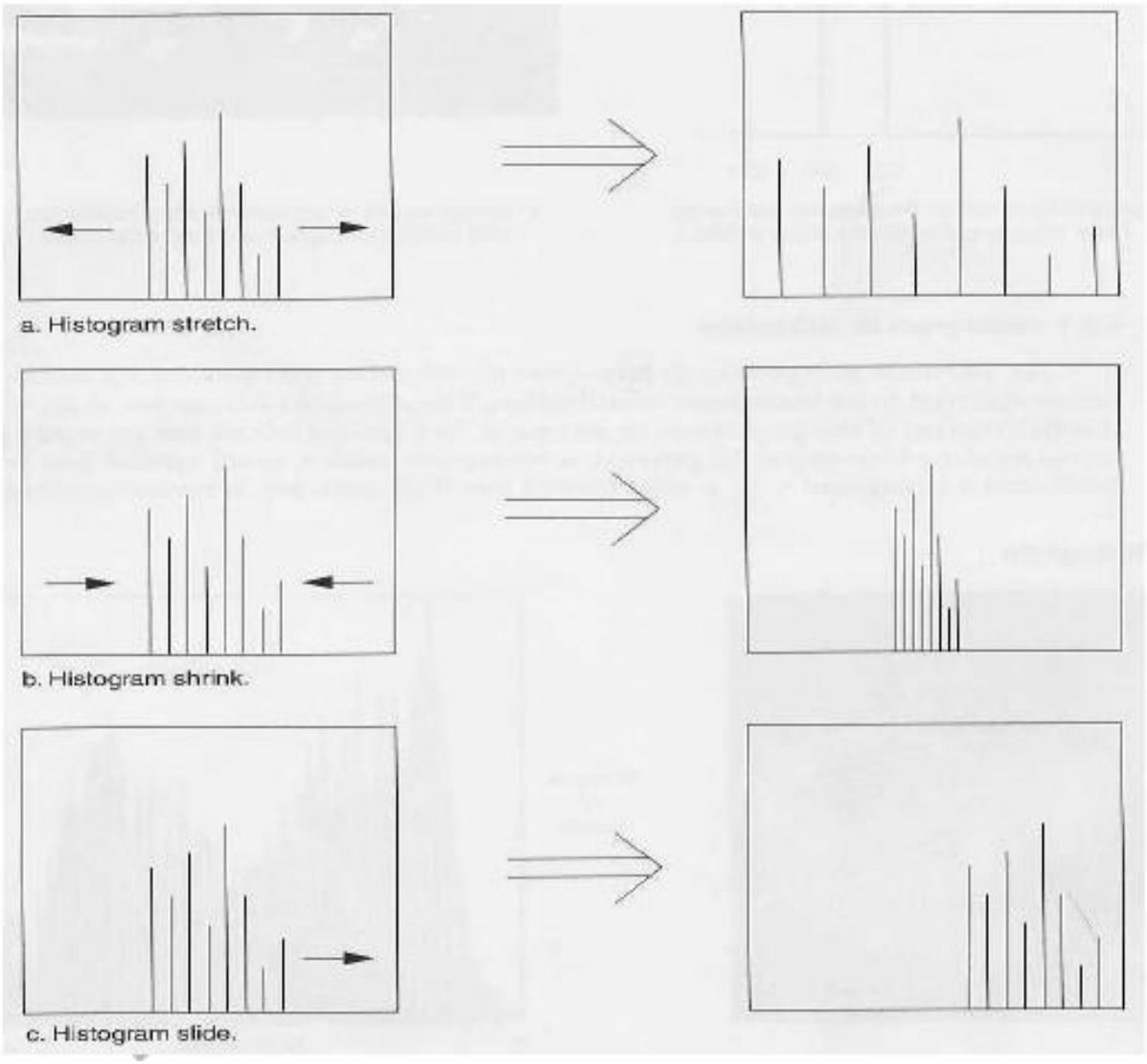
3. Histogram with two major peaks, called bimodal (**contrast with the background)**

Sub Image and Its Histogram



Histogram Modification

The histogram can be Enhanced or modified by mapping functions, which will stretch, shrink (compress), or slide the histogram.



Applications of Histograms

1. Calculations in software.
2. Analyze an image.
3. Properties predicted by histogram.
4. Brightness adjusted using histogram.
5. Contrast adjusted by histogram.
6. Image equalization.
7. Produce a high contrast image.
8. Used in thresholding to improve appearance of image.
7. Determine the type of transformation is applied in the algorithm.

Histogram Processing

Intensities of L level digital image

r_k K=0,1,2,…,L-1

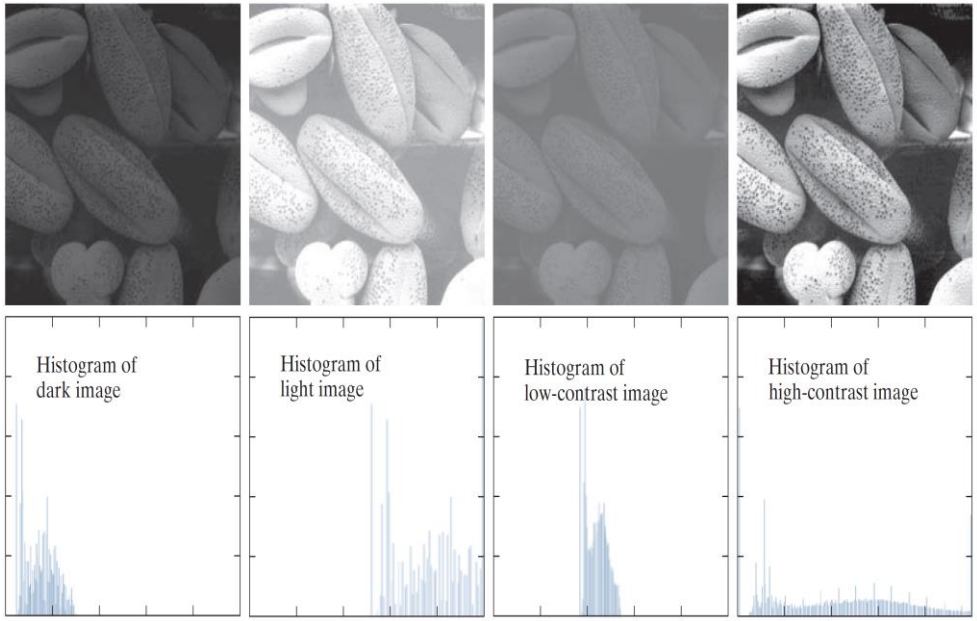
No. of pixels in f with intensities rk

Unnormalized histogram

$$h(r_k) = n_k$$

histogram bins

subdivisions of the intensity scale



normalized histogram of *f* is

$$p(r_k) = \frac{h(r_k)}{MN} = \frac{n_k}{MN}$$

Image Intensities as Random Variable

$$p(z_k) = \frac{n_k}{MN}$$

$$\sum_{k=0}^{L-1} p(z_k) = 1$$

Intensities of $M \times$N

$k = 0,1,2, \ldots, L-1$

number of times

total number of pixels

mean (average) intensity:

$$m = \sum_{k=0}^{L-1} z_k p(z_k)$$

variance of the intensities:

$$\sigma^2 = \sum_{k=0}^{L-1} (z_k - m)^2 p(z_k)$$

Image Intensities as Random Variable

The variance is a measure of the spread of the values of z about the mean, so it is a useful measure of image contrast. In general, the nth central moment of random variable z about the mean is defined as,

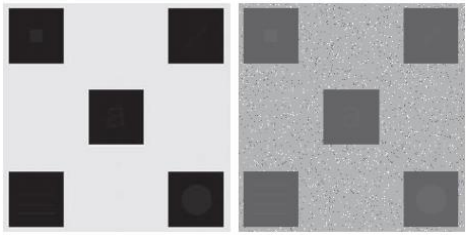
Mean: $$\mu_n(z) = \sum_{k=0}^{L-1} (z_k - m)^n p(z_k)$$

$$\mu_0(z) = 1 \qquad \mu_1(z) = 0 \qquad \mu_2(z) = \sigma^2$$

e.g., a positive third moment indicates that the intensities are biased to values higher than the mean, a negative third moment would indicate the opposite condition, and a zero third moment would tell us that the intensities are distributed approximately equally on both sides of the mean.

Histogram Statistics for Image Enhancement

52	55	61
59	57	54
53	58	60



List all pixels:

[52, 55, 61, 59, 57, 54, 53, 58, 60]

a. Minimum = 52

b. Maximum = 61

c. Mean:

$$\mu = (52 + 55 + 61 + 59 + 57 + 54 + 53 + 58 + 60)/9$$

$$\approx 56.56$$

$$\sigma = \sqrt{\frac{1}{9}\sum(xi - \mu)^2} = \sqrt{\frac{81.20}{9}} \approx 3.00$$

Histogram Stretching

$$I_{new}(r,c) = \left[\frac{I(\mathrm{r,c}) - I_{min}(r,c)}{I_{max}(r,c) - I_{min}(r,c)}\right] \times (\mathrm{Max} - \mathrm{Min}) + \mathrm{Min}$$

Where,

*I(r,c)*max is the largest gray- level in the image I(r,c).

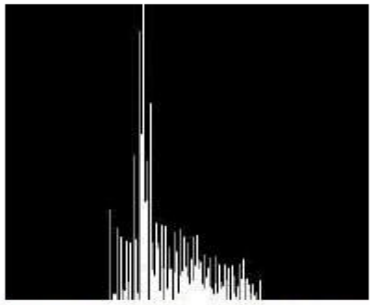
*I(r,c)*min is the smallest gray- level in the image I(r,c).

$$I_{new}(r,c) = \left[\frac{I(\mathrm{r,c}) - I_{min}(r,c)}{I_{max}(r,c) - I_{min}(r,c)}\right] \times 255$$

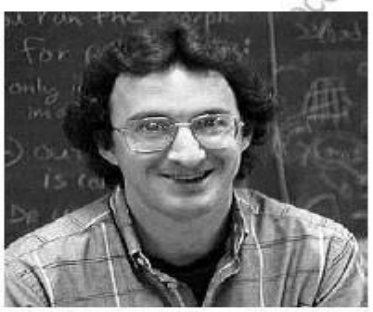
MAX and MIN correspond to the maximum and minimum gray – level values possible (for an 8-bit image these are 255 and 0). This equation will take an image and stretch the histogram across the entire gray-level range which has the effect of increasing the contrast of a low contrast image.



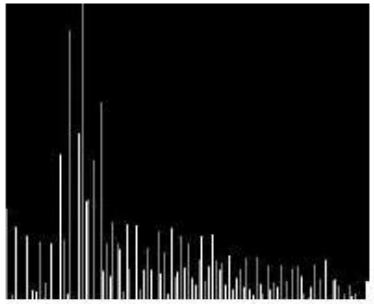
a. Low-contrast image



b. Histogram of low-contrast image



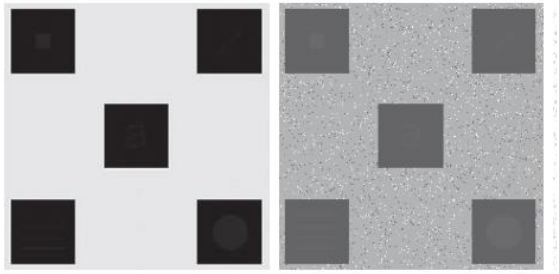
c. Image after histogram stretching



d. Histogram of image after stretching

Enhancement using Contrast Stretching

52	55	61
59	57	54
53	58	60



a. Minimum = 52

b. Maximum = 61

$C. Range = 9$

Pixel	$I_{new} = \frac{I - min}{\max - min} \times 255$	Rounded
52	$\frac{52-52}{61-52} \times 255 = 0$	0
53	$\frac{53-52}{61-52} \times 255 = 28.33$	28
54	56.67	57
55	85.0	85
57	141.67	142
58	170.0	170
59	198.33	198
60	226.67	227
61	255	255

Enhancement using Contrast Stretching

Pixel	$I_{new} = \frac{I - min}{\max - min} \times 255$	Rounded
52	$\frac{52-52}{61-52} \times 255 = 0$	0
53	$\frac{53-52}{61-52} \times 255 = 28.33$	28
54	56.67	57
55	85.0	85
57	141.67	142
58	170.0	170
59	198.33	198
60	226.67	227
61	255	255

52	55	61
59	57	54
53	58	60



0	**85**	**255**
198	**142**	**57**
28	**170**	**227**

Enhancement using Contrast Stretching

52	55	61
59	**57**	54
53	58	60

0	**85**	**255**
198	**142**	**57**
28	**170**	**227**

Histogram Before vs After:

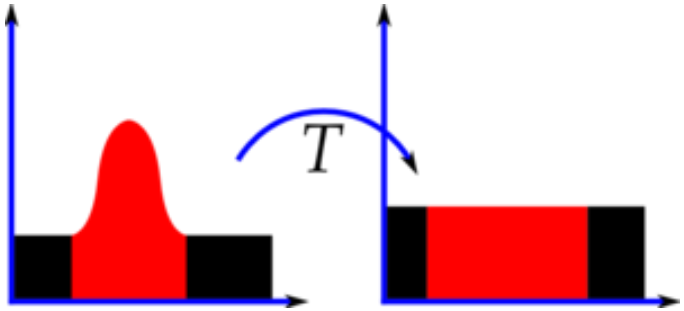
Value Range	Original Counts	Enhanced Value Range
52–61	Mostly dark	Spread to 0–255

Contrast is **significantly improved** — pixels span entire range now.

Histogram Equalization

- Histogram equalization is a method of contrast adjustment.
- It is a technique of the resultant image is as flat as possible With histogram stretching.





Adaptive Histogram Equalization

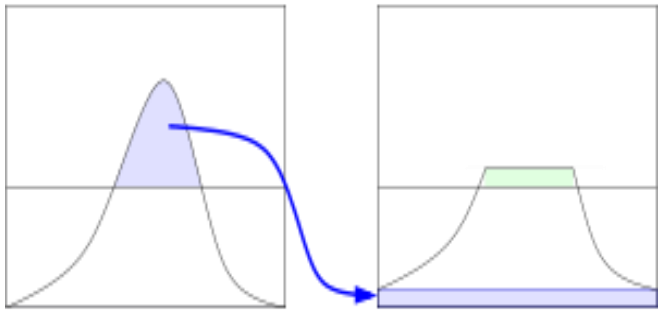
- Adaptive histogram equalization (AHE)
  - ✓Improve contrast in images,
  - ✓Computes several histograms,
  - ✓Correspond to a distinct section of image,
  - ✓Redistribute the lightness values of image
  - ✓Improving the local contrast and enhancing the definitions of edges in each region of an image.
  - ✓Tendency to overamplify noise in relatively homogeneous regions of an image.

- *Contrast limiting adaptive histogram equalization* (CLAHE) prevents this by limiting the amplification.

Contrast Limited AHE (CLAHE)

- First developed for use in aircraft cockpit displays.
- AHE may cause noise to be amplified in near-constant regions. Contrast Limited AHE (CLAHE) is limited to reduce the problem of noise amplification.
- contrast amplification is given by the slope of the transformation function.

- Slope of the neighborhood CDF to the value of the histogram at that pixel value.



Histogram Equalization

- Let a discrete grayscale image {*x*} and
- n_i : number of occurrences of gray level *i*.
- The probability of an occurrence of a pixel of level *i* in the image is

$$p_x(i) = p(x = i) = \frac{n_i}{n}, \quad 0 \leq i < L$$

- L be in the total number of gray levels in the image (typically 256), *n* being the total number of pixels in the image, and px(i) being in fact the image's histogram for pixel value i, normalized to [0,1].

Histogram Equalization with Intensities Mapping

intensity mappings

$s = T(r)$

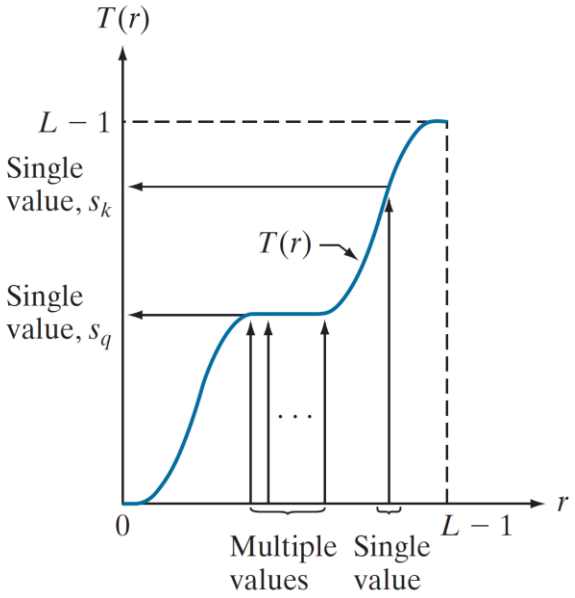
$0 \leq r \leq L-1$

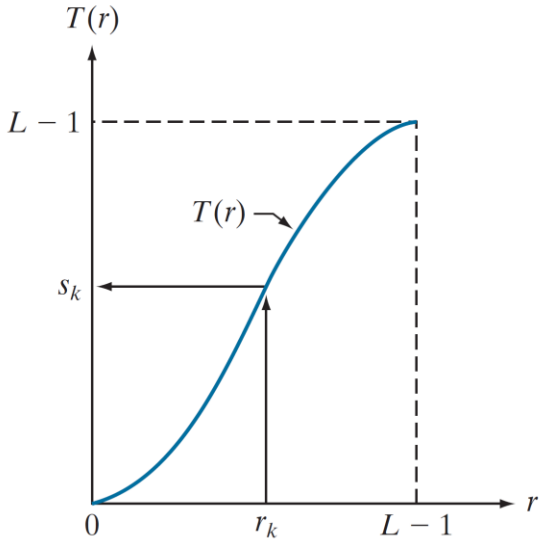
Dark $r = 0$

white $r = L-1$

multiple values can map to a single value.

one-to-one mapping, both ways.





Histogram Equalization with PDF & Intensities

intensity mappings

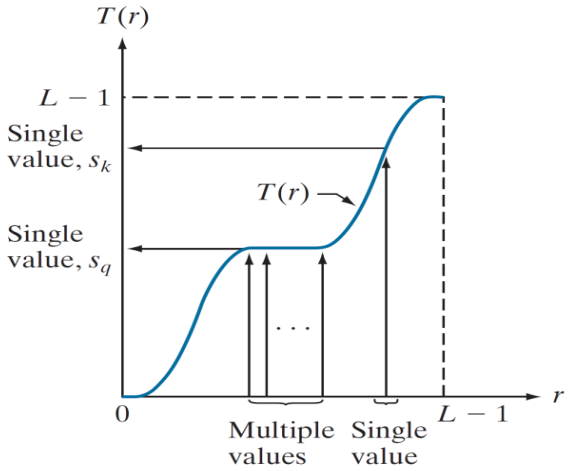
$0 \leq T(r) \leq L-1$

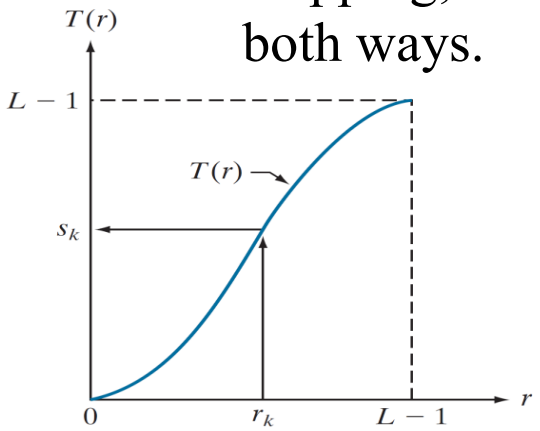
$0 \leq r \leq L-1$

Inverse transformation

$r = T^{-1}(s)$

$0 \leq s \leq L-1$





one-to-one mapping, both ways.

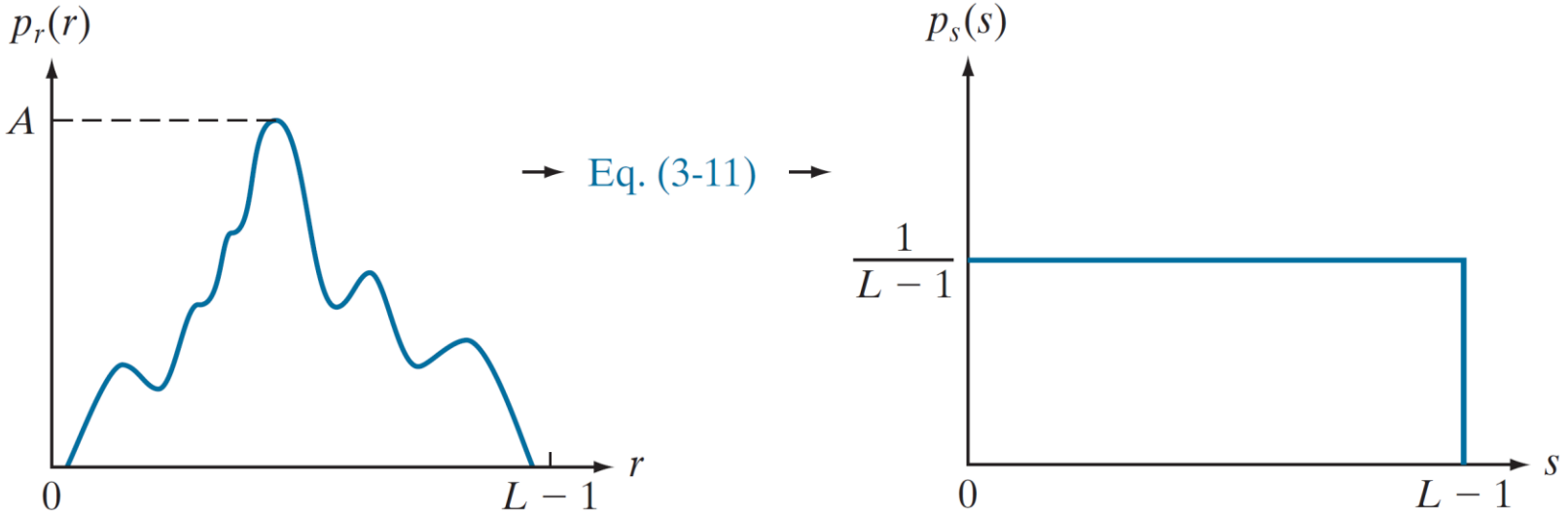
$T(r)$ be monotonically increasing guarantees that output intensity values will never be less than corresponding input values

guarantees that the range of output intensities is the same as the input

Histogram Equalization with PDF & Intensities

PDFs of intensity values r

PDFs of intensity values s



The PDF of the transformed (mapped) variable s can be obtained as

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right|$$

Dummy variable of integration

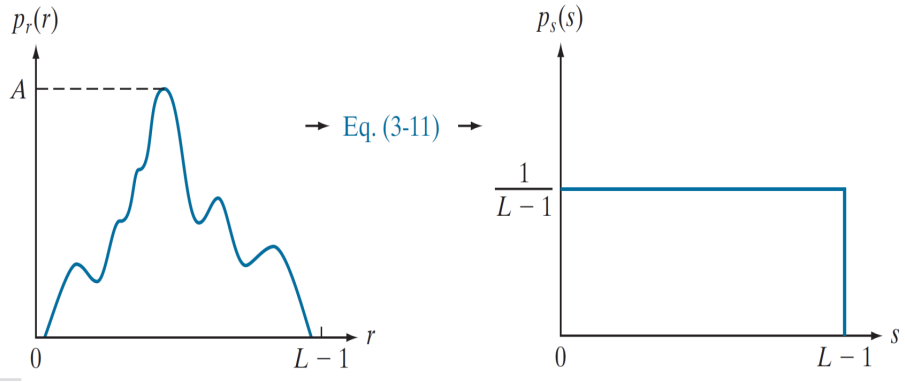
transformation function of particular importance in image processing

$$s = T(r) = (L-1)\int_0^r p_r(\omega)\,d\omega$$

Histogram Equalization with PDF & Intensities

$$s = T(r) = (L-1)\int_0^r p_r(\omega)\,d\omega$$

CDF of random variable r



- the area under the function cannot decrease as r increases.
- When the upper limit is $r = (L-1)$, the integral evaluates to 1, as it for PDF.
- maximum value of s is $L-1$

$$\frac{ds}{dr} = \frac{dT(r)}{dr} = (L-1)\frac{d}{dr}\left[\int_0^r p_r(\omega)\,d\omega\right] = (L-1)\,p_r(r)$$

Leibniz's rule in calculus that the derivative of a definite integral with respect to its upper limit is the integrand evaluated at the limit.

Histogram Equalization with PDF & Intensities

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right|$$

$$= p_r(r)\left|\frac{1}{(L-1)\,p_r(r)}\right|$$

$$= \frac{1}{(L-1)}$$

Uniform PDF

Histogram Equalization

Example: Histogram Equalization

Suppose that the (continuous) intensity values in an image have the PDF.

$$p_r(r) = \begin{cases} \dfrac{2r}{(L-1)^2} & for\ 0 \le r \le L-1 \\ 0 & otherwise \end{cases}$$

$$s = T(r) = (L-1)\int_0^r p_r(w)\,dw = \frac{2}{L-1}\int_0^r w\,dw = \frac{r^2}{L-1}$$

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right| = \frac{2r}{(L-1)^2}\left|\left[\frac{ds}{dr}\right]^{-1}\right|$$

$$= \frac{2r}{(L-1)^2}\left|\left[\frac{d}{dr}\frac{r^2}{L-1}\right]^{-1}\right| = \frac{2r}{(L-1)^2}\left|\frac{(L-1)}{2r}\right| = \frac{1}{L-1}$$

The last step follows because r is nonnegative and $L > 1$. As expected, the result is a uniform PDF.

Steps for Histogram Equalization

1. Read an image
2. Find the total number of pixels (n) in the image

   n=row*column.
3. Find the number of pixels(n_k) for each gray label(r_k).
4. Find PDF for each gray label of the image.

$$p_r(r_k) = \frac{n_k}{n} = \frac{n_k}{MN} \qquad r_k \in [0, L-1]$$

5. Find CDF and multiply this by L-1.

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_k) \quad ; k = 0,1,2,\ldots,L-1$$

6. Finally round of s_k to its nearest integer.

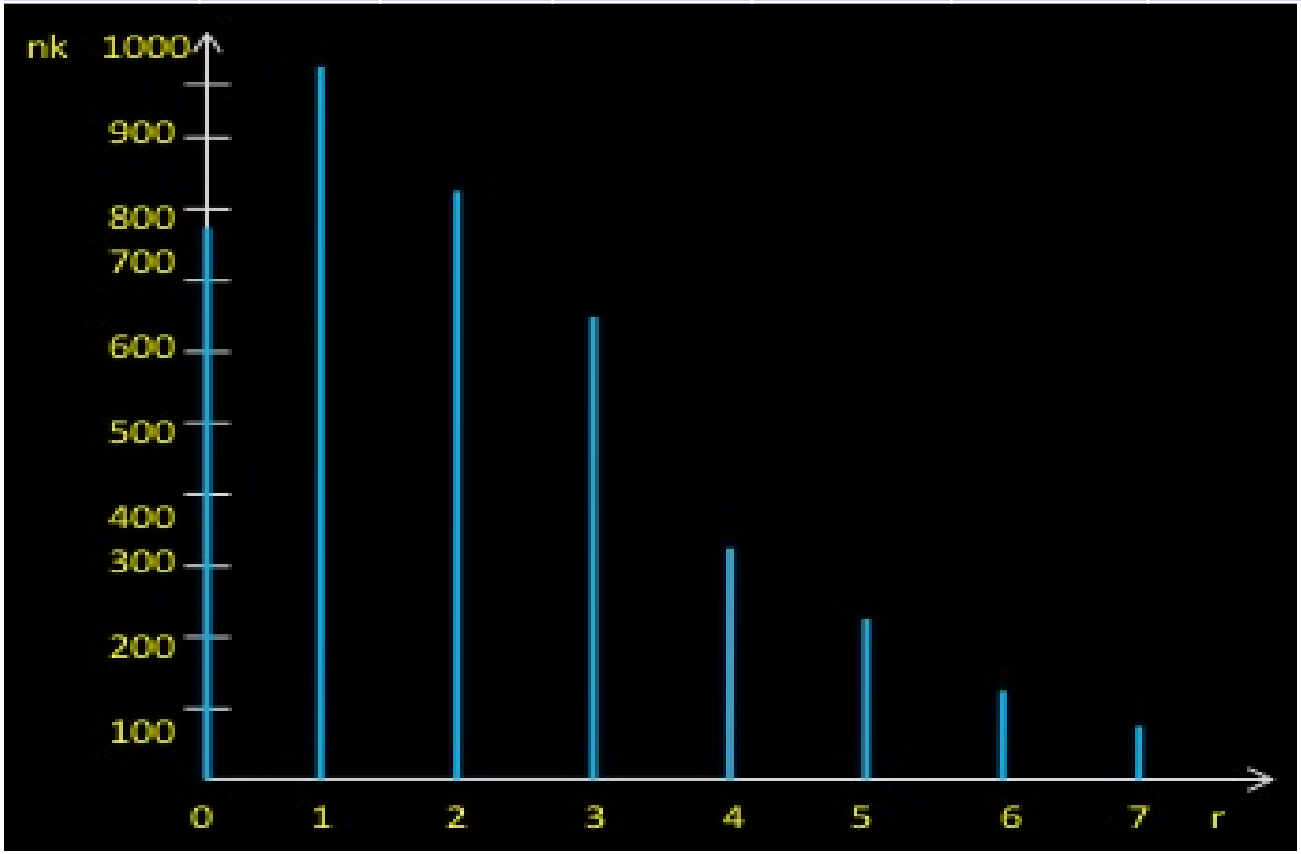
Example1: Histogram Equalization

$$I_{original} = \begin{bmatrix} 52 & 55 & 61 & 59 \\ 79 & 61 & 76 & 61 \\ 85 & 104 & 94 & 85 \\ 59 & 71 & 85 & 69 \end{bmatrix} \Rightarrow I_{Equalized} = \begin{bmatrix} 16 & 32 & 112 & 64 \\ 176 & 112 & 160 & 112 \\ 223 & 255 & 239 & 223 \\ 64 & 144 & 223 & 128 \end{bmatrix}$$

Intensity (Gray Level, r)	Count Intensity, n_k	Probability (pdf), $p_r(r_k) = \frac{n_k}{MN}$	CDF $= \sum p_r(r_k)$	$s_k = (L-1) \times CDF$ (Rounding value)
52	1	1/16 = 0.0625	0.0625	255 × 0.0625 = 16
55	1	0.0625	0.0625 + 0.0625 = 0.125	255 × 0.125 = 32
59	2	2/16 = 0.125	0.125 + 0.125 = 0.25	255 × 0.25 = 64
61	3	3/16 = 0.1875	0.4375	255 × 0.4375= 112
69	1	0.0625	0.5	255 × 0.5= 128
71	1	0.0625	0.5625	255 × 0.5625 = 144
76	1	0.0625	0.625	255 × 0.625 = 160
79	1	0.0625	0.6875	255 × 0.6875 = 176
85	3	3/16 = 0.1875	0.875	255 × 0.875= 223
94	1	0.0625	0.9375	255 × 0.9375 = 239
104	1	0.0625	1.0	255 × 1.0 = 255
	16			

Example2: Histogram Equalization

Gray Levels (r)	0	1	2	3	4	5	6	7
No. of Pixels	790	1023	850	656	329	245	122	81

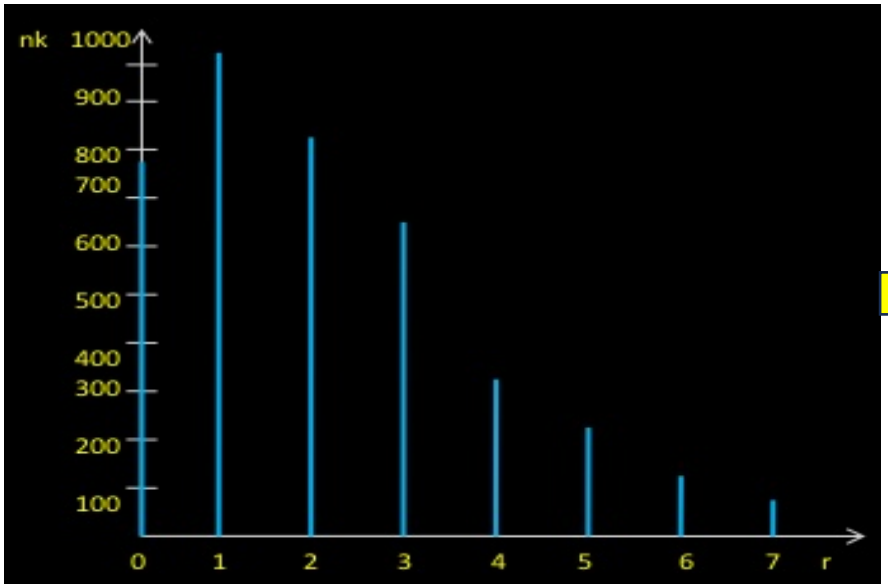


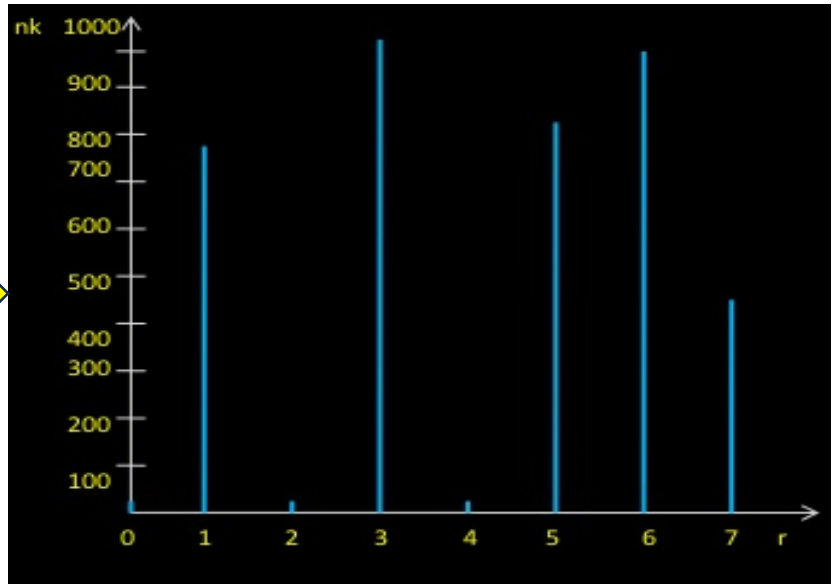
Example: Mechanics of Histogram Equalization

Intensity (Gray Level, r)	Count Intensity, n_k	Probability, $p_r(r_k) = \frac{n_k}{MN}$	CDF $= \sum p_r(r_k)$	$s_k = (L-1) \times CDF$	Rounding off
0	790	790/4096= 0.19	0.19	$(8-1) \times 0.19 = 1.33$	1
1	1023	1023/4096=0.25	0.44	3.08	3
2	850	0.21	0.65	4.55	5
3	656	0.16	0.81	5.67	6
4	329	0.08	0.89	6.23	6
5	245	0.06	0.95	6.65	7
6	122	0.03	0.98	6.86	7
7	81	0.02	1	7	7
	$MN = 4096$	$\sum p_r(r_k) = 1$			

Histogram Equalization

Gray Levels (r)	0	1	2	3	4	5	6	7
No. of Pixels	790	1023	850	656	329	245	122	81
Equalized pixels	0	790	0	1023	0	850	656+329 = 985	245+122+81 = 484





Thank You

Lecture 2.2

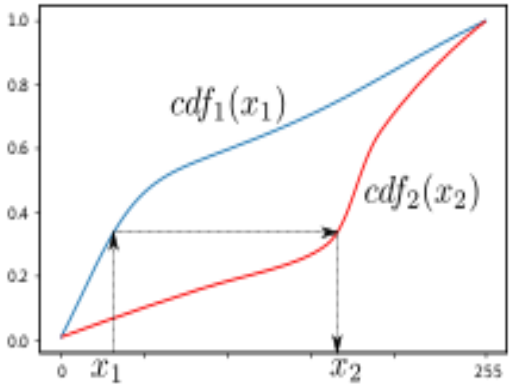
Course Name: Digital Image Processing
Course code: 4245

Histogram Matching

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Histogram Matching

- Unify contrast level of a group of images.
- Histogram equalization is taken as histogram matching.
- Modify the histogram of an input image to be similar to the **normal distribution**.
- First equalize the histogram of both images.
- Need to map each pixel using the equalized histograms. Then modify each pixel.



Histogram Matching

Intensity (Gray Level, r_k)	0	1	2	3	4	5	6	7
Ref. Image No. of pixels (Count Intensity), n_k	0	0	0	0	20	20	16	8
Input Image No. of pixels (Count Intensity), n_k	8	10	10	2	12	16	4	2

Histogram Matching

Reference Image					
Intensity (Gray Level, r_k)	No. of pixels (Count Intensity), n_k	Probability, $p_r(r_k) = \frac{n_k}{MN}$	CDF $= \sum p_r(r_k)$	$s_k = (L-1) \times CDF$	Rounding off of s_k
0	0	0/64=0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	20	20/64=0.3125	0.3125	(8-1)*0.3125=2.1875	2
5	20	0.3125	0.625	4.375	4
6	16	0.25	0.875	6.125	6
7	8	0.125	1	7	7
	$MN = 64$	$\sum p_r(r_k) = 1$			

Histogram Matching

Input Image

Intensity (Gray Level, r_k)	No. of pixels (Count Intensity), n_k	Probability, $p_r(r_k) = \frac{n_k}{MN}$	CDF $= \sum p_r(r_k)$	$H_k = (L-1) \times CDF$	Rounding off of H_k
0	8	8/64=0.125	0.125	(8-1)*0.125=0.875	1
1	10	10/64=0.15625	0.28125	1.96875	2
2	10	0.15625	0.4375	3.0625	3
3	2	0.03125	0.46875	3.28125	3
4	12	2/64=0.1875	0.65625	4.59375	5
5	16	0.25	0.90625	6.34375	6
6	4	0.0625	0.96875	6.78125	7
7	2	0.03125	1	7	7
	$MN = 64$	$\sum p_r(r_k) = 1$			

Histogram Matching

Intensity (Gray Level, r_k)	H_k	s_k	Map	No. of pixels (Count Intensity), n_k	No. of pixels (Count Intensity), q_k
0	1	0	4	8	0
1	2	0	4	10	0
2	3	0	5	10	0
3	3	0	5	2	0
4	5	2	6	12	18
5	6	4	6	16	12
6	7	6	7	4	28
7	7	7	7	2	6

Intensity (Gray Level, r_k)	0	1	2	3	4	5	6	7
Ref. Image, n_k	0	0	0	0	20	20	16	8
Input Image, n_k	8	10	10	2	12	16	4	2
Matched (Mapping)	4	4	5	5	6	6	7	7

Histogram Matching

Source Image A: [0,0,1,1,2,2,2] ,
Target Image B: [0,1,1,1,2,2,2]

Intensity (Gray Level , r_k)	No. of pixels (Count Intensity), n_k		Probability, $p_r(r_k) = \frac{n_k}{MN}$		CDF $= \sum p_r(r_k)$		Mapping
	A	B	A	B	A	B	
0	2	1	0.29	0.14	0.29	0.14	CDF 0.29 (A) is closer to 0.57 (B) → match to $r_k = 1$
1	2	3	0.29	0.43	0.58	0.57	CDF 0.58 (A) is closer to 1.0 (B) → match to $r_k = 2$
2	3	3	0.43	0.43	1.0	1.0	CDF 1.0 (A) is same as 1.0 (B) → match to $r_k = 2$
	$MN = 7$	$MN = 7$	$\sum p_r(r_k) = 1$				

Histogram Matching

Source Image A: [0,0,1,1,2,2,2],
Target Image B: [0,1,1,1,2,2,2]

Original A	Matched Image, A' (Mapping)	Gray Level	Count	Probability
0	1	1	2	2/7=0.29
0	1			
1	2	2	5	5/7=0.71
1	2			
2	2			
2	2			
2	2			

Histogram Matching

Source: A=[52, 55, 61, 59, 79]; Target: B=[55, 59, 59, 61, 87]

Intensity (Gray Level, r_k)	No. of pixels (Count Intensity), n_k		Probability, PDF $p_r(r_k) = \frac{n_k}{MN}$		CDF $= \sum p_r(r_k)$		Mapping
	A	B	A	B	A	B	
52	1	0	0.2	0	0.2	0	CDF 0.2(A) is equal to 0.2(B) → match to target level=55
55	1	1	0.2	0.2	0.4	0.2	CDF 0.4(A) is closer to 0.6(B) → match to target level =59
59	1	2	0.2	0.4	0.6	0.6	CDF 0.6(A) is equal to 0.6(B) → match to target level =59
61	1	1	0.2	0.2	0.8	0.8	CDF 0.8(A) is equal to 0.8(B) → match to target level =61
79	1	0	0.2	0	1.0	1.0	CDF 1.0(A) is equal to 1.0(B) → match to target level =87
87	0	1	0	0.2	1.0	1.0	CDF 1.0(A) is equal to 1.0(B) → match to target level =87
	$MN = 5$	$MN = 5$	$\sum p_r(r_k) = 1$				

Histogram Matching

Source: A=[52, 55, 61, 59, 79]; Target: B=[55, 59, 59, 61, 87]

Mapped values using histogram matching:

Original source A	52	55	61	59	79
Matched Image, A' (Mapping)	55	59	61	59	87

Thank You

Lecture 2.3

Course Name: Digital Image Processing
Course code: 4245

Spatial Filtering, Correlation & Convolution, Kernel

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Spatial Correlation and Convolution

These are fundamental operations in image filtering, edge detection, feature extraction, CNNs, etc.

- Correlation → similarity measurement
- Convolution → system response

Correlation

$$(w \star f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

180°

Convolution

$$(w \star f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x-s, y-t)$$

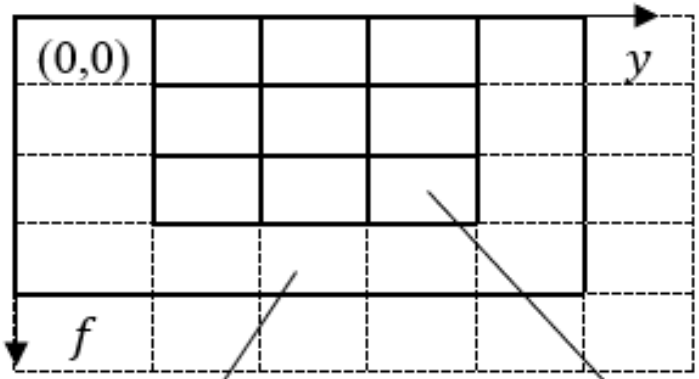
Real Applications

- Edge detection
- Gaussian blur
- Sobel filter
- Laplacian filter
- CNN models like OpenAI vision systems

In CNN, convolution is actually **correlation** (kernel is not flipped).

Mechanics of Linear Spatial Filtering

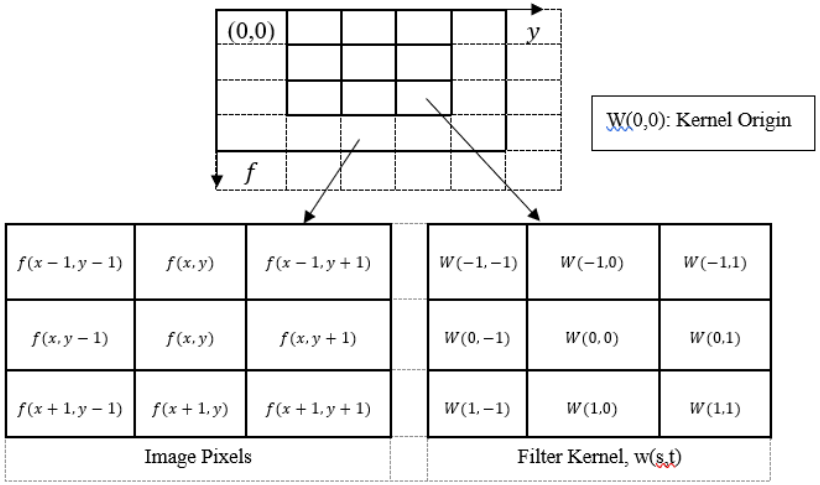
Mechanics of linear spatial filtering using a 3x3 kernel.



W(0,0): Kernel Origin

$f(x-1,y-1)$	$f(x,y)$	$f(x-1,y+1)$	$W(-1,-1)$	$W(-1,0)$	$W(-1,1)$
$f(x,y-1)$	$f(x,y)$	$f(x,y+1)$	$W(0,-1)$	$W(0,0)$	$W(0,1)$
$f(x+1,y-1)$	$f(x+1,y)$	$f(x+1,y+1)$	$W(1,-1)$	$W(1,0)$	$W(1,1)$
Image Pixels			Filter Kernel, w(s,t)		

Mechanics of Linear Spatial Filtering



$$g(x,y) = w(-1,-1)f(x-1,y-1) + w(-1,0)f(x-1,y) + \ldots$$
$$+ w(0,0)f(x,y) + \ldots + w(1,1)f(x+1,y+1)$$

Response image

Image

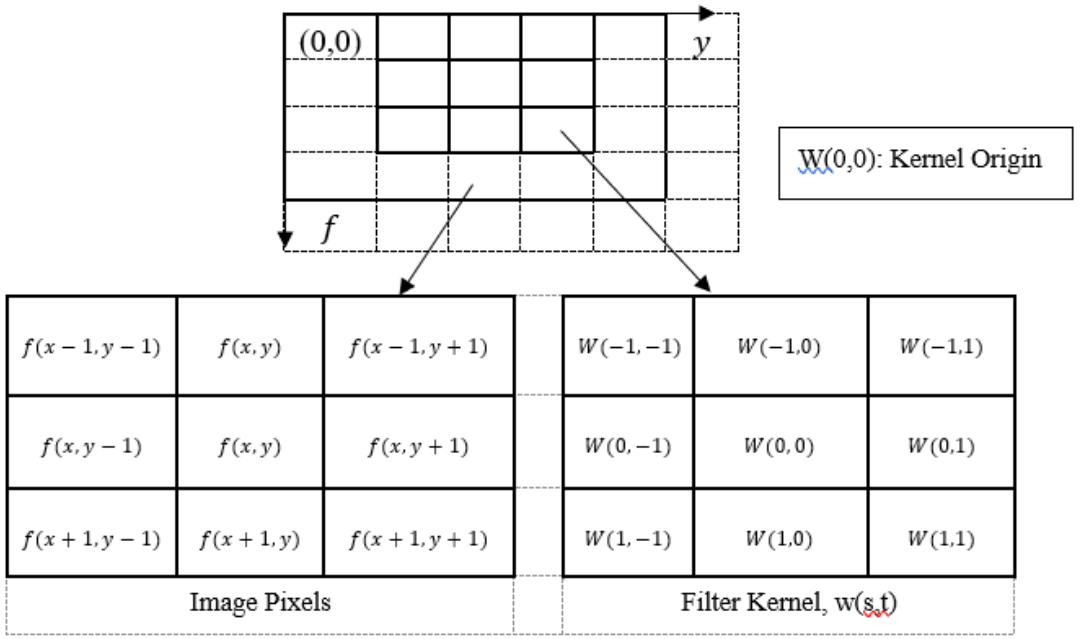
Filter kernel/ kernel (mask, template, window)

Kernel moves from pixel to pixel

Spatial Correlation and Convolution

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t) \quad \Rightarrow \quad G = \omega \circ f$$

Correlation consists of moving the center of a kernel over an image, and computing the sum of products at each location.



The mechanics of *spatial convolution* are the same, except that the correlation kernel is rotated by 180^0.

Spatial Correlation and Convolution

$x = 0, 1, 2, 3, 4, 5, 6, 7$

0	$g(0) = \sum_{s=-2}^{2} w(s)f(s+0) = 0 \times 0 = 0$
1	$g(1) = \sum_{s=-2}^{2} w(s)f(s+1) = 8 \times 1 = 8$
2	$g(2) = \sum_{s=-2}^{2} w(s)f(s+2) = 2 \times 1 = 2$
3	$g(3) = \sum_{s=-2}^{2} w(s)f(s+3) = 4 \times 1 = 4$
4	$g(4) = \sum_{s=-2}^{2} w(s)f(s+4) = 2 \times 1 = 2$
5	$g(5) = \sum_{s=-2}^{2} w(s)f(s+5) = 1 \times 1 = 1$
6	$g(6) = \sum_{s=-2}^{2} w(s)f(s+6) = 0 \times 0 = 0$
7	$g(7) = \sum_{s=-2}^{2} w(s)f(s+7) = 0 \times 0 = 0$

$\therefore$ Correlation value $= 0, 8, 2, 4, 2, 1, 0, 0$

$\Rightarrow$ Extended (Full) Correlation value $= 0, 0, 0, 8, 2, 4, 2, 1, 0, 0, 0, 0$

$\therefore$ Convolution value $= 0, 1, 2, 4, 2, 8, 0, 0$

$\Rightarrow$ Extended (Full) Convolution value $= 0, 0, 0, 1, 2, 4, 2, 8, 0, 0, 0, 0$

Correlation and Convolution of Kernel

Correlation

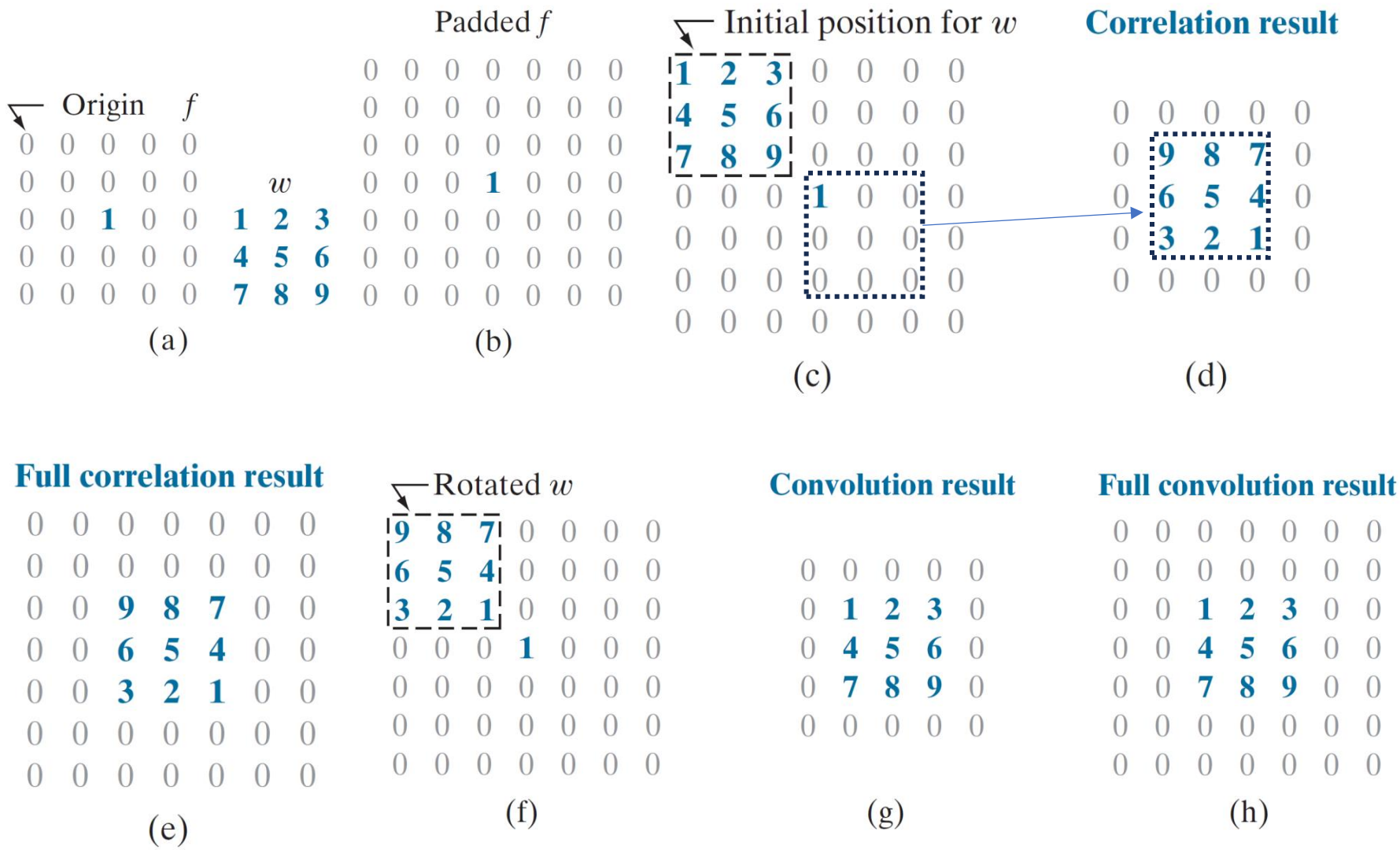
$$(w \star f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

180°

Convolution

$$(w \star f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x-s, y-t)$$

Correlation and Convolution of Kernel



Correlation and Convolution Array Size

Array Size: $S_v \times S_h$

Kernel size: $m \times n$

Image size: $M \times N$

$$S_v = m + M - 1$$

$$S_h = n + N - 1$$

Properties of Correlation and Convolution:

Property	Convolution	Correlation
Commutative	$f \star g = g \star f$	—
Associative	$f \star (g \star h) = (f \star g) \star h$	—
Distributive	$f \star (g + h) = (f \star g) + (f \star h)$	$f \star (g + h) = (f \star g) + (f \star h)$

Image Convolution Applications

100	100	100
100	0	0
100	0	0

Target: Detect vertical edges using the Sobel kernel for vertical edges:

$$K = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix}$$

At (1,1) and compute convolution sum.

$$S = (100 \times 1) + (100 \times 0) + (100 \times -1) + (100 \times 2) + (0 \times 0) + (0 \times -2) + (100 \times 1) + (0 \times 0) + (0 \times -1) = 300$$

Example: Correlation on 2D image with 2D Filter Mask

Input Image f

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

Correlation Kernel, w

1	-1	-1
1	2	-1
1	1	1

Example: Correlation on 2D image with 2D Filter Mask

Correlation:

1	-1	-1		
1	4	-2	2	3
1	2	1	3	3
	2	2	1	2
	1	3	2	2

$g(1,1) = 4 - 2 + 2 + 1 = 5$

1	-1	-1		
2	4	-2	3	
2	1	3	3	
2	2	1	2	
1	3	2	2	

$g(1,2) = 2 + 4 - 2 + 2 + 1 + 3 = 10$

	1	-1	-1	
2	2	4	-3	
2	1	3	3	
2	2	1	2	
1	3	2	2	

$g(1,3) = 2 + 4 - 3 + 1 + 3 + 3 = 10$

		1	-1	-1
2	2	2	6	-1
2	1	3	3	1
2	2	1	2	
1	3	2	2	

$g(1,4) = 2 + 6 + 3 + 3 = 14$

1	-2	-2	2	3
1	4	-1	3	3
1	2	2	1	2
	1	3	2	2

$g(2,1) = -2 - 2 + 4 - 1 + 2 + 2 = 3$

2	-2	-2	3	
2	2	-3	3	
2	2	1	2	
1	3	2	2	

$g(2,2) = 2 - 2 - 2 + 2 + 2 - 3 + 2 + 2 + 1 = 4$

2	2	-2	-3	
2	1	6	-3	
2	2	1	2	
1	3	2	2	

$g(2,3) = 2 - 2 - 3 + 1 + 6 - 3 + 2 + 1 + 2 = 6$

2	2	2	-3	-1
2	1	3	6	-1
2	2	1	2	1
1	3	2	2	

$g(2,4) = 2 - 2 + 3 + 6 + 1 + 2 = 11$

Example: Correlation on 2D image with 2D Filter Mask

	2	2	2	3
1	-2	-1	3	3
1	4	-2	1	2
1	1	3	2	2

$g(3,1) = -2 - 1 + 4 - 2 + 1 + 3 = 3$

2	2	2	3	
2	-1	-3	3	
2	4	-1	2	
1	3	2	2	

$g(3,2) = 2 - 1 - 3 + 2 + 4 - 1 + 1 + 3 + 2 = 9$

2	2	-2	-3	
2	1	-3	-3	
2	2	2	-2	
1	3	2	2	

$g(3,3) = 1 - 3 - 3 + 2 + 2 - 2 + 3 + 2 + 2 = 4$

2	2	2	-3	
2	1	3	-3	-1
2	2	1	2	-1
1	3	2	2	1

$g(3,4) = 3 - 3 + 1 + 2 + 2 + 2 = 7$

	2	2	2	3
	2	1	3	3
1	-2	-2	1	2
1	2	-3	2	2
1	1	1		

$g(4,1) = -2 - 2 + 2 - 3 = -5$

2	2	2	3	
2	1	3	3	
2	-2	-1	2	
1	6	-2	2	
1	1	1		

$g(4,2) = 2 - 2 - 1 + 1 + 6 - 2 = 4$

2	2	2	3	
2	1	3	3	
2	2	-1	-2	
1	3	2		
	1	1		

$g(4,3) = 2 - 1 - +2 + 2 = 6$

2	2	2	3	
2	1	3	3	
2	2	1	-2	-1

Output Image, g:

5	10	10	14
3	4	6	11
3	9	4	7
-5	4	6	5

Example: Convolution on 2D image with 2D Filter Mask

Input Image f

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

Convolution Kernel, w

1	-1	-1
1	2	-1
1	1	1

Convolution Kernel (w) must be rotated by 180°. Thus,

1	-1	-1
1	2	-1
1	1	1

180° ⟷

1	1	1
-1	2	1
-1	-1	1

Example: Convolution on 2D image with 2D Filter Mask

Convolution:

1	1	1		
-1	4	2	2	3
-1	-2	1	3	3
	2	2	1	2
	1	3	2	2

$g(1,1) = 4 + 2 - 2 + 1 = 5$

1	1	1		
-2	4	2	3	
-2	-1	3	3	
2	2	1	2	
1	3	2	2	

$g(1,2) = 4$

	1	1	1	
2	-2	4	3	
2	-1	-3	3	
2	2	1	2	
1	3	2	2	

$g(1,3) = 4$

		1	1	1
2	2	-2	6	1
2	1	-3	-3	1
2	2	1	2	
1	3	2	2	

$g(1,4) = -2$

1	2	2	2	3
-1	4	1	3	3
-1	-2	2	1	2
	1	3	2	2

$g(2,1) = 9$

2	2	2	3	
-2	2	3	3	
-2	-2	1	2	
1	3	2	2	

$g(2,2) = 6$

2	2	2	3	
2	-1	6	3	
2	-2	-1	2	
1	3	2	2	

$g(2,3) = 14$

2	2	2	3	1
2	1	-3	6	1
2	2	-1	-2	1
1	3	2	2	

$g(2,4) = 5$

	2	2	2	3
1	2	1	3	3
-1	4	2	1	2
-1	-1	3	2	2

$g(3,1) = 11$

2	2	2	3	
2	1	3	3	
-2	4	1	2	
-1	-3	2	2	

$g(3,2) = 7$

2	2	2	3	
2	1	3	3	
2	-2	2	2	
1	-3	-2	2	

$g(3,3) = 6$

2	2	2	3	
2	1	3	3	1
2	2	-1	4	1
1	3	-2	-2	1

$g(3,4) = 5$

	2	2	2	3
	2	1	3	3
1	2	2	1	2
-1	2	3	2	2
-1	-1	1		

$g(4,1) = 9$

2	2	2	3	
2	1	3	3	
2	2	1	2	
-1	6	2	2	
-1	-1	1		

$g(4,2) = 3$

2	2	2	3	
2	1	3	3	
2	2	1	2	
1	-3	4	2	
	-1	-1	1	

$g(4,3) = 8$

2	2	2	3	
2	1	3	3	
2	2	1	2	1
1	3	-2	4	1
		-1	-1	1

$g(4,4) = 5$

Convolution

5	4	4	-2
9	6	14	5
11	7	6	5
9	12	8	5

Final output Image, *g*

Example: Filtering Using Convolution Process

Example: Using the following 3×3 mask to perform the convolution process on the shaded pixels in the 5×5 input image below. Write the filtered image.

5×5 Input Image f

30	40	50	70	90
40	50	80	60	100
35	255	70	0	120
30	45	80	100	130
40	50	90	125	140

3×3 mask

0	1/6	0
1/6	1/3	1/6
0	1/6	0

30	40	50
40	50	80
35	255	70

40	50	70
50	80	60
255	70	0

50	70	90
80	60	100
70	0	120

40	50	80
35	255	70
30	45	80

50	80	60
255	70	0
45	80	100

80	60	100
70	0	120
80	100	130

35	255	70
30	45	80
40	50	90

255	70	0
45	80	100
50	90	125

70	0	120
80	100	130
90	125	140

Example: Filtering Using Convolution Process

5×5 Input Image f

30	40	50	70	90
40	50	80	60	100
35	255	70	0	120
30	45	80	100	130
40	50	90	125	140

3×3 mask

0	1/6	0
1/6	1/3	1/6
0	1/6	0

30	40	50
40	50	80
35	255	70

40	50	70
50	80	60
255	70	0

50	70	90
80	60	100
70	0	120

40	50	80
35	255	70
30	45	80

50	80	60
255	70	0
45	80	100

80	60	100
70	0	120
80	100	130

35	255	70
30	45	80
40	50	90

255	70	0
45	80	100
50	90	125

70	0	120
80	100	130
90	125	140

1. $0 \times 30 + \frac{1}{6} \times 40 + 0 \times 50 + \frac{1}{6} \times 40 + \frac{1}{3} \times 50 + \frac{1}{6} \times 80 + 0 \times 35 + \frac{1}{6} \times 255 + 0 \times 70 = 85$
2. $0 \times 40 + \frac{1}{6} \times 50 + 0 \times 70 + \frac{1}{6} \times 50 + \frac{1}{3} \times 80 + \frac{1}{6} \times 60 + 0 \times 255 + \frac{1}{6} \times 70 + 0 \times 0 = 65$
3. $0 \times 50 + \frac{1}{6} \times 70 + 0 \times 90 + \frac{1}{6} \times 80 + \frac{1}{3} \times 60 + \frac{1}{6} \times 100 + 0 \times 70 + \frac{1}{6} \times 0 + 0 \times 120 = 61$
4. $0 \times 40 + \frac{1}{6} \times 50 + 0 \times 80 + \frac{1}{6} \times 35 + \frac{1}{3} \times 255 + \frac{1}{6} \times 70 + 0 \times 30 + \frac{1}{6} \times 45 + 0 \times 80 = 118$
5. $0 \times 50 + \frac{1}{6} \times 80 + 0 \times 60 + \frac{1}{6} \times 255 + \frac{1}{3} \times 70 + \frac{1}{6} \times 0 + 0 \times 45 + \frac{1}{6} \times 80 + 0 \times 100 = 92$
6. $0 \times 80 + \frac{1}{6} \times 60 + 0 \times 100 + \frac{1}{6} \times 70 + \frac{1}{3} \times 0 + \frac{1}{6} \times 120 + 0 \times 80 + \frac{1}{6} \times 100 + 0 \times 130 = 58$
7. $0 \times 35 + \frac{1}{6} \times 255 + 0 \times 70 + \frac{1}{6} \times 30 + \frac{1}{3} \times 45 + \frac{1}{6} \times 80 + 0 \times 40 + \frac{1}{6} \times 50 + 0 \times 90 = 84$
8. $0 \times 255 + \frac{1}{6} \times 70 + 0 \times 0 + \frac{1}{6} \times 45 + \frac{1}{3} \times 80 + \frac{1}{6} \times 100 + 0 \times 50 + \frac{1}{6} \times 90 + 0 \times 125 = 77$
9. $0 \times 70 + \frac{1}{6} \times 0 + 0 \times 120 + \frac{1}{6} \times 80 + \frac{1}{3} \times 100 + \frac{1}{6} \times 130 + 0 \times 90 + \frac{1}{6} \times 125 + 0 \times 140 = 89$

Filtered image

30	40	50	70	90
40	**85**	**65**	**61**	100
35	**118**	**92**	**58**	120
30	**84**	**77**	**89**	130
40	50	90	125	140

Convolution Sum

Why Convolution is Important?

- Image smoothing
- Sharpening
- Edge detection
- CNN operations
- Gaussian blur
- Motion blur

Superposition summation: $$y[n] = \sum_{k=-\infty}^{\infty} x[k]h_k[n]$$

Convolution sum: $$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]. \quad -\infty < n < \infty$$

Convolution notation: $$y[n] = x[n] * h[n]$$

Convolution Sum

$$n = -1, 0, 1, 2, 3$$

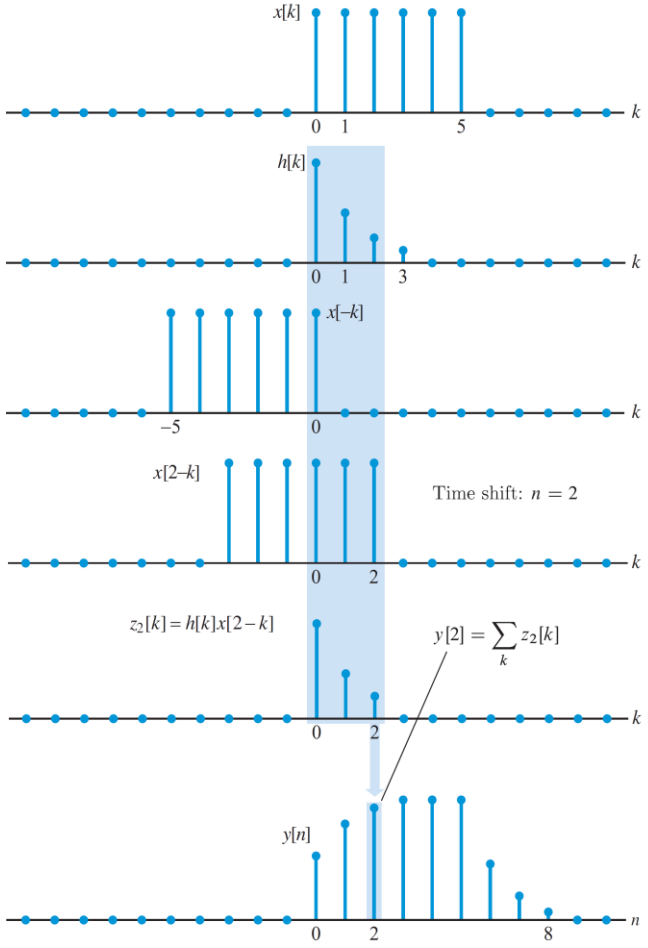
$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]. \quad -\infty < n < \infty$$

$$\begin{aligned}
y[-1] &= \cdots + x[-1]h[0] + x[0]h[-1] + x[1]h[-2] + x[2]h[-3] + \cdots \\
y[0] &= \cdots + x[-1]h[1] + x[0]h[0] + x[1]h[-1] + x[2]h[-2] + \cdots \\
y[1] &= \cdots + x[-1]h[2] + x[0]h[1] + x[1]h[0] + x[2]h[-1] + \cdots \\
y[2] &= \cdots + x[-1]h[3] + x[0]h[2] + x[1]h[1] + x[2]h[0] + \cdots \\
y[3] &= \cdots + x[-1]h[4] + x[0]h[3] + x[1]h[2] + x[2]h[1] + \cdots
\end{aligned}$$

Convolution Sum

$$x[n] = \{\underset{\uparrow}{1}\ 1\ 1\ 1\ 1\ 1\},\quad h[n] = \{\underset{\uparrow}{1}\ 0.5\ 0.25\ 0.125\}$$

h(n)	1	0.5	0.25	0.125
X(n)				
1	1	0.5	0.25	0.125
1	1	0.5	0.25	0.125
1	1	0.5	0.25	0.125
1	1	0.5	0.25	0.125
1	1	0.5	0.25	0.125
1	1	0.5	0.25	0.125

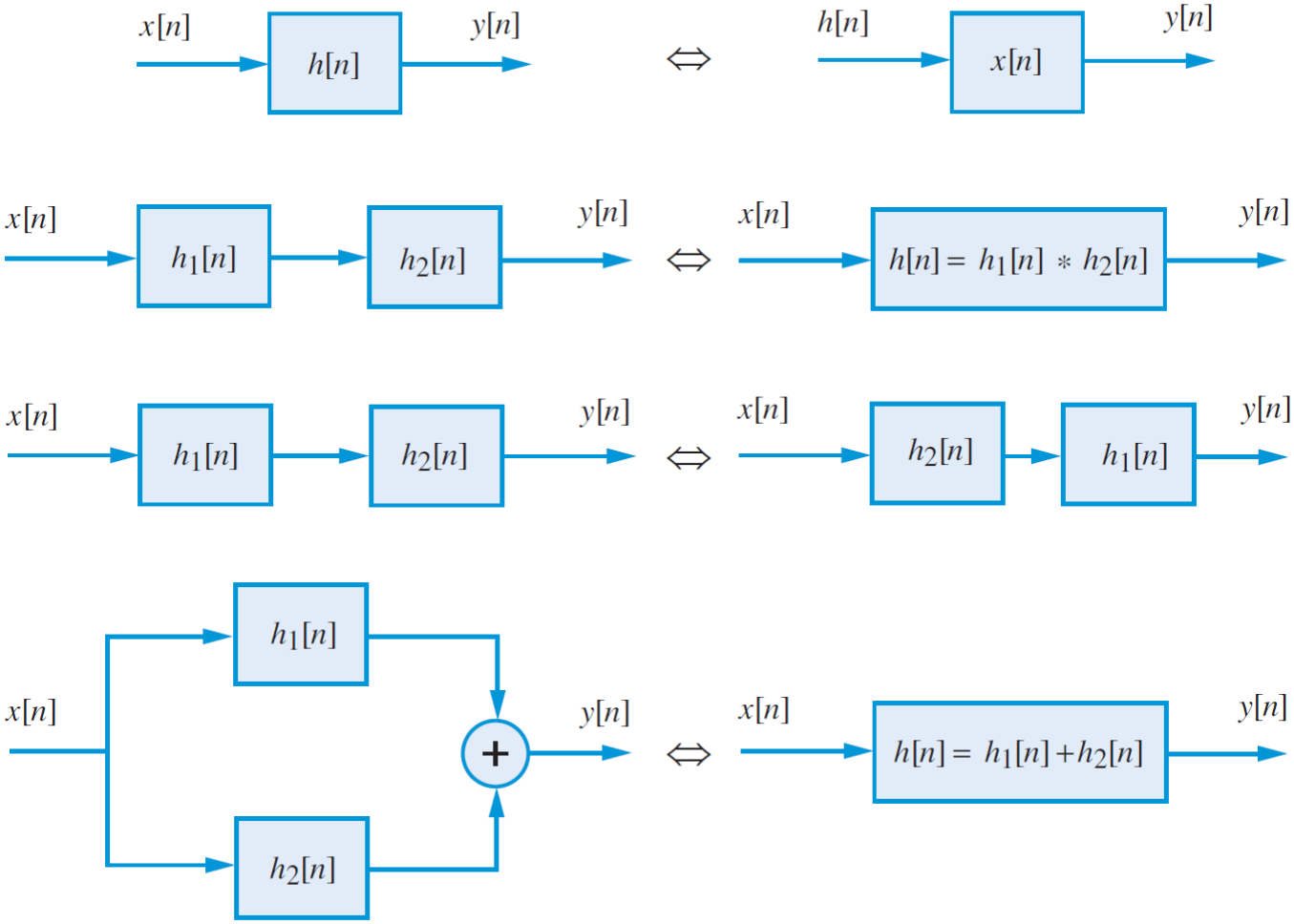


y(n)={1, 1+0.5, 1+0.5+0.25,
1+0.5+0.25+0.125, 1+0.5+0.25+0.125,
1+0.5+0.25+0.125, 0.5+0.25+0.125,
0.5+0.25, 0.125 }

$$y[n] = \{\underset{\uparrow}{1}, 1.5, 1.75, 1.875, 1.875, 1.875, 0.875, 0.375, 0.125\}$$

Properties of LTI System

$$h[n] * x[n] = x[n] * h[n]$$



Properties of LTI System

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k] = \sum_{m=-\infty}^{\infty} h[m]x[n-m] = h[n] * x[n].$$

$$v[n] = \sum_{k=-\infty}^{\infty} x[k]h_1[n-k] \quad \text{and} \quad y[n] = \sum_{m=-\infty}^{\infty} h_2[m]v[n-m]$$

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] \sum_{m=-\infty}^{\infty} h_2[m]h_1[(n-k)-m]$$

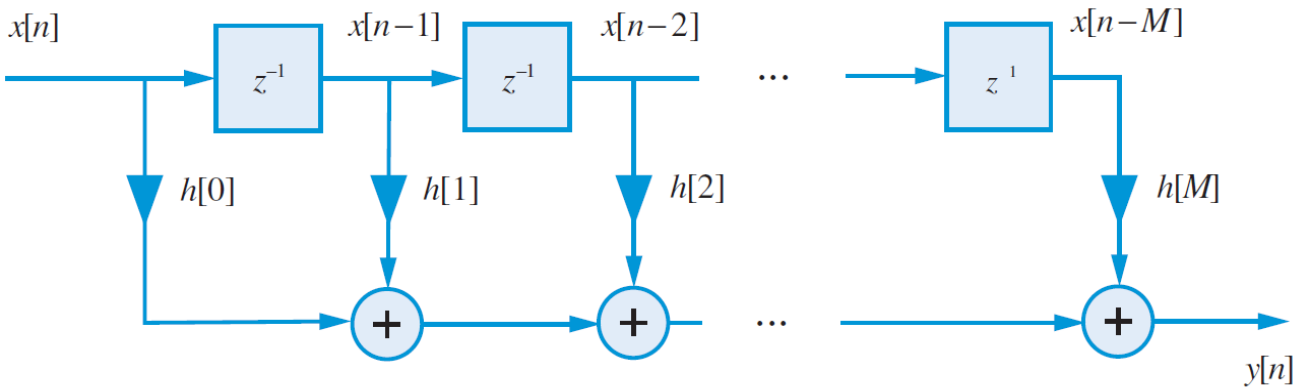
Properties of LTI System

Property	Formula
Identity	$x[n] * \delta[n] = x[n]$
Delay	$x[n] * \delta[n - n_0] = x[n - n_0]$
Commutative	$x[n] * h[n] = h[n] * x[n]$
Associative	$(x[n] * h_1[n]) * h_2[n] = x[n] * (h_1[n] * h_2[n])$
Distributive	$x[n] * (h_1[n] + h_2[n]) = x[n] * h_1[n] + x[n] * h_2[n]$

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k] = x[n] * h[n]$$

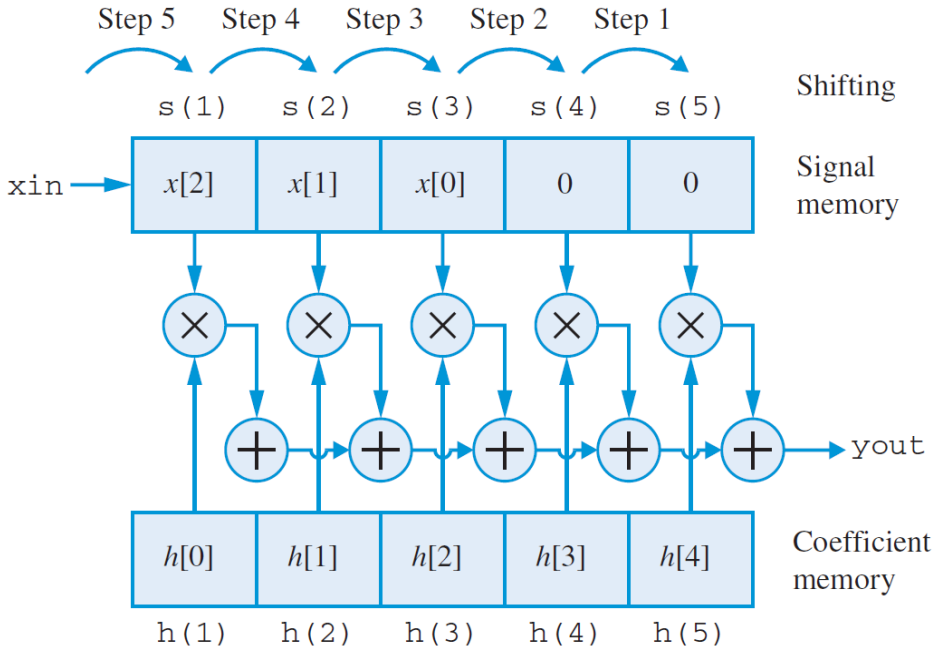
$$y[n] = h_1[n] * x[n] + h_2[n] * x[n] = (h_1[n] + h_2[n]) * x[n] \triangleq h[n] * x[n]$$

FIR



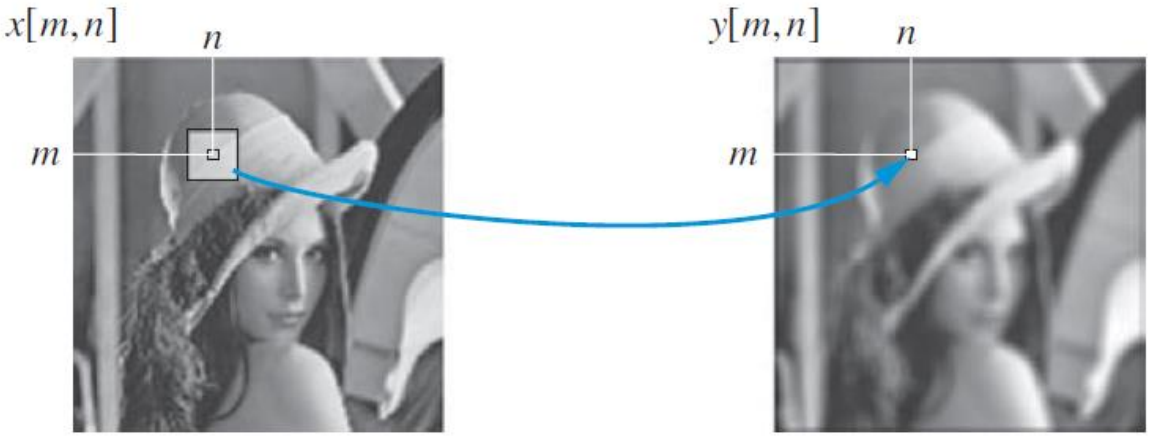
FIR system (filter) has a **finite number of non-zero coefficients/**finite neighborhood of pixels.

In image processing, most spatial filters (like smoothing, sharpening) are **FIR filters**.



FIR

Consider a 3×3 region around the pixel $x[m, n]$. Then the smoothed pixel value $y[m, n]$ can be computed as an arithmetic mean of the nine pixels in the local region



$$
\begin{aligned}
y[m,n] = \tfrac{1}{9}(x[m-1,n-1] &+ x[m-1,n] &+ x[m-1,n+1] \\
+ x[m,n-1] &+ x[m,n] &+ x[m,n+1] \\
+ x[m+1,n-1] &+ x[m+1,n] &+ x[m+1,n+1])
\end{aligned}
$$

FIR

$$y[m,n] = \tfrac{1}{9}(x[m-1,n-1] + x[m-1,n] + x[m-1,n+1] + x[m,n-1] + x[m,n] + x[m,n+1] + x[m+1,n-1] + x[m+1,n] + x[m+1,n+1])$$

$$y[m,n] = \sum_{k=-1}^{1} \sum_{\ell=-1}^{1} \left(\frac{1}{9}\right) x[m-k,n-\ell]$$

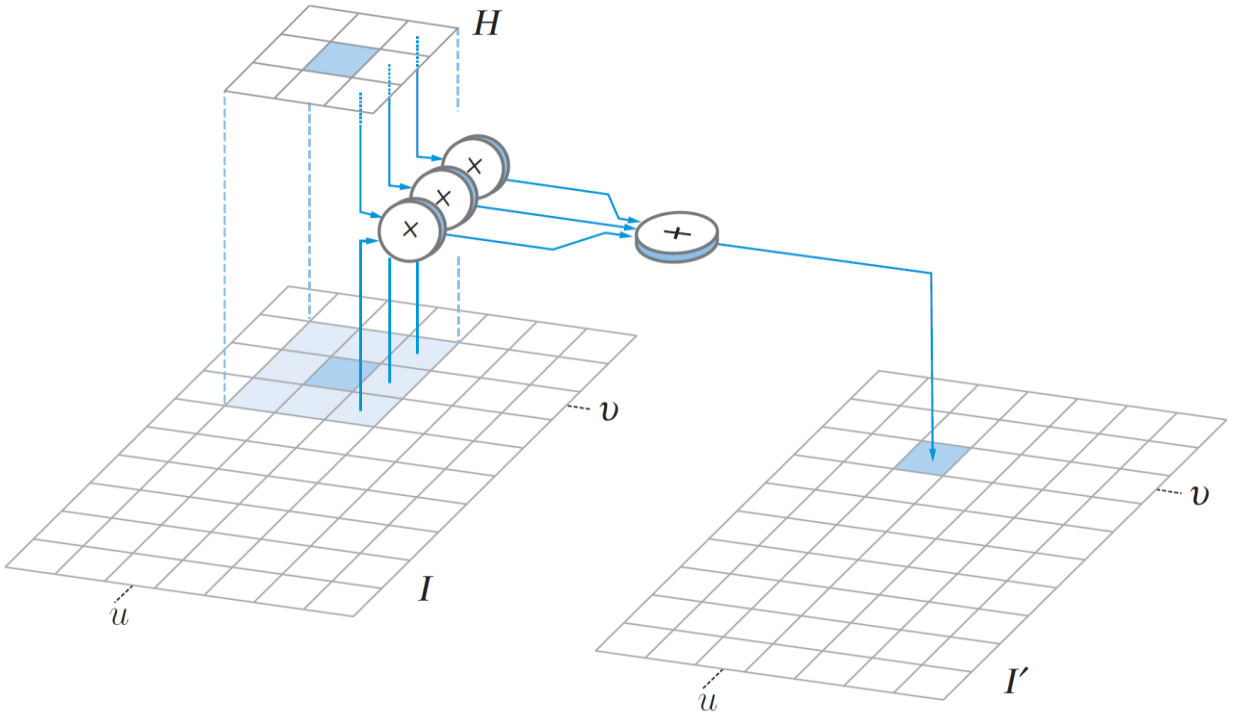
$$h[m,n] = \begin{cases} \frac{1}{9}, & -1 \le m,n \le 1 \\ 0, & \text{otherwise} \end{cases}$$

$$y[m,n] = \sum_{k=-1}^{1} \sum_{\ell=-1}^{1} h[k,\ell]x[m-k,n-\ell]$$

FIR

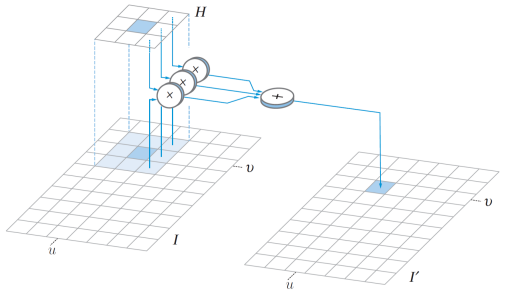
which is a 2D convolution of image $x[m, n]$ with an FIR spatial filter $h[m, n]$. A general expression for 2D convolution, when the FIR spatial filter has finite symmetric support $(2K + 1) \times (2L + 1)$, is given by

$$y[m, n] = \sum_{k=-K}^{K} \sum_{\ell=-L}^{L} h[k, \ell] x[m - k, n - \ell]$$



FIR

$$y[m,n] = \sum_{k=m-K}^{m+K} \sum_{\ell=n-L}^{n+L} x[k,\ell]h[m-k,n-\ell]$$



Summary:

1. The filter array $h[k,l]$ is rotated by 180° to obtain $h[-k,-l]$.
2. The rotated array is moved over the image so that the origin $h[0,0]$ coincides with the current image pixel $x[m,n]$.
3. All filter coefficients are multiplied with the corresponding image pixels and the results are added.
4. The resulting sum is stored at the current pixel $x[m,n]$ in the filtered image $y[m,n]$.

Thank You

Lecture 2.4

Course Name: Digital Image Processing
Course code: 4245

Spatial Filtering and Kernels for Smoothing Intensities

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Spatial Filtering

Spatial filtering means:
Modifying pixel values using **neighboring pixels directly in the image domain**.

It operates on the image $f(x, y)$ itself (not in frequency domain).

General Formula

For a filter (mask/kernel) $w(s,t)$ of size $m \times n$:

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

Spatial vs Frequency Domain

Spatial Domain	Frequency Domain
Direct pixel operation	Uses Fourier Transform
Intuitive	Mathematical
Small kernels	Global filtering

Types of Spatial Filters

(A) Smoothing Filters (Low-Pass): Averaging Filter (3×3), Gaussian Filter

(B) Sharpening Filters (High-Pass): Laplacian Filter, Sobel Operator

Spatial Filtering

Classification on the basis of linearity: There are two types,

1. Linear Spatial Filter

2. Non-linear Spatial Filter

General Classification:

Smoothing Spatial Filter: Used for blurring and noise reduction in the image. Blurring is pre-processing steps for removal of small details and Noise Reduction is accomplished by blurring.

Types of Smoothing Spatial Filter:

1. Linear Filter (Mean Filter)

2. Order Statistics (Non-linear) filter

Sharpening Spatial Filter

Sharpening Spatial Filters

- Sharpening aims to highlight fine details (e.g. edges) in an image, or enhance detail that has been blurred through errors or imperfect capturing devices.
- Image blurring can be achieved using averaging filters, and hence sharpening can be achieved by operators that invert averaging operators.
- In mathematics, averaging is equivalent to the concept of integration, and differentiation inverts integration.
- Thus, sharpening spatial filters can be represented by partial derivatives.

Sharpening Spatial Filter

Smoothing Spatial Filters

- Used for blurring and for noise reduction.
- Blurring is used in preprocessing steps to remove small details from an image prior to (large) object extraction,
- Bridge small gaps in lines or curves.
- Noise reduction can be accomplished by blurring with a linear filter and also by nonlinear filtering.

Sharpening Spatial Filter

It is also known as derivative filter. The purpose of the sharpening spatial filter is just the opposite of the smoothing spatial filter. Its main focus in on the removal of blurring and highlight the edges. It is based on the first and second order derivative.

First order derivative:

- Must be zero in flat segments.
- Must be non zero at the onset of a grey level step.
- Must be non zero along ramps.

First order derivative in 1-D:

$$f' = f(x+1) - f(x)$$

Second order derivative:

- Must be zero in flat areas.
- Must be zero at the onset and end of a ramp.
- Must be zero along ramps.

Second order derivative in 1-D:

$$f'' = f(x+1+f(x-1)) - 2f(x)$$

1st Order Partial Derivatives of Digital Functions

The first order partial derivatives of the digital image $f(x, y)$ are:

$$\frac{\partial f}{\partial x} = f(x + 1, y) - f(x, y) \; and \; \frac{\partial f}{\partial y} = f(x, y + 1) - f(x, y)$$

The first derivative must be:

1) zero along flat segments (i.e. constant gray values).

2) non-zero at the outset of gray level step or ramp (edges or noise)

3) non-zero along segments of continuing changes (i.e. ramps).

2nd Order Partial Derivatives of Digital Functions

The second order partial derivatives of the digital image $f(x, y)$ are:

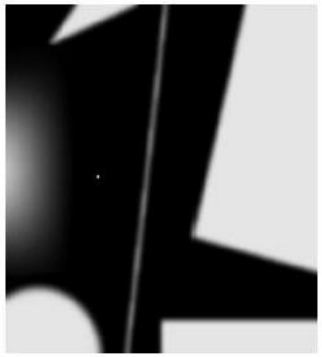
$$\frac{\partial^2 f}{\partial y^2} = f(x, y + 1) + f(x, y - 1) - 2f(x, y)$$
$$\frac{\partial^2 f}{\partial x^2} = f(x + 1, y) + f(x - 1, y) - 2f(x, y)$$

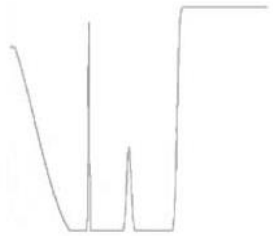
The second derivative must be:

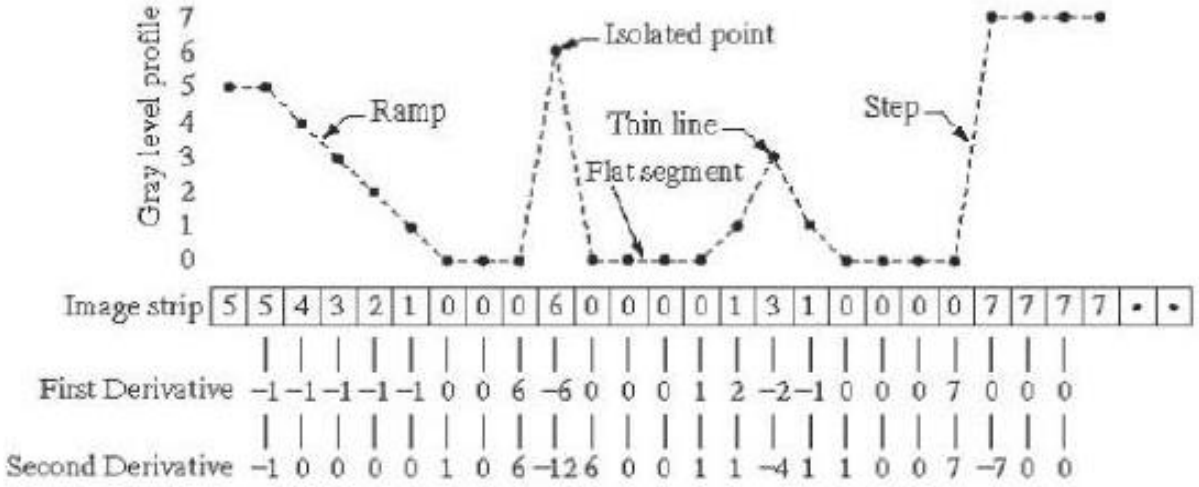
1) zero along flat segments.

2) nonzero at the outset and end of a gray-level step or ramp;
3) Zero along ramps

Example of Partial Derivatives







- 1st derivative detects thick edges while 2nd derivative detects thin edges.
- 2nd derivative has much stronger response at gray-level step than 1st derivative.

Thus, we can expect a second-order derivative to enhance fine detail (thin lines, edges, including noise) much more than a first-order derivative.

Mean Filtering for Image Processing

Both mean and median filters are spatial domain filters used to:

- Reduce noise
- Smooth images
- Improve visual quality
- But they differ in how they process pixel values.

Mean Filtering

Mean filtering replaces each pixel value with the average (arithmetic mean) of its neighborhood. It's a linear filter (uses convolution).

For a $M \times N$ neighborhood $W(x, y)$:

$$g(x, y) = \frac{1}{M \times N} \sum_{(s,t) \in W(x,y)} f(s, t)$$

Where, $f(x, y) \rightarrow$ Original pixel intensity, and $g(x, y) \rightarrow$ Filtered pixel intensity.

Mean Filtering for Image Processing

Effect

- Smooths image and reduce Gaussian noise.
- Blurs edges and fine details.

Advantages

- Simple and fast to compute.
- Effective for random (Gaussian) noise.

Disadvantages

- Blurs sharp edges.
- Poor for salt-and-pepper noise.

Mean Filter

Types of Mean filter:

(i) Averaging filter: Used in reduction of the detail in image. All coefficients are equal.

(ii) Weighted averaging filter: pixels are multiplied by different coefficients. Center pixel is multiplied by a higher value than average filter.

3x3 Averaging Linear Filters

Averaging linear filtering of an image f of size $M \times N$ with a filter mask of size $m \times n$ is given by the expression:

$$g(x,y) = \frac{\sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t) f(x+s, y+t)}{\sum_{s=-a}^{a}\sum_{t=-b}^{b} w(s,t)}$$

To generate a complete filtered image this equation must be applied for $x = 0,1,2,\ldots,M-1$ & $y = 0,1,2,\ldots,N-1$.

$\frac{1}{9} \times$

1	1	1
1	1	1
1	1	1

Standard average filter

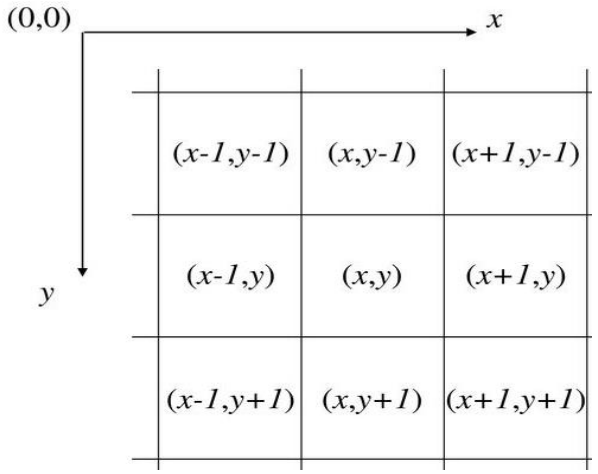
$\frac{1}{16} \times$

1	2	1
2	4	2
1	2	1

Weighted average filter

Averaging Filter an Intuitive Approach

Replace each pixel by the average of pixels in a square window surrounding this pixel,



$$g(m, n) = \frac{1}{9}\big(f(m, -1n - 1)\,\big(f(m, -1n - 1) + + j\,f\,(m, -1n) + f\,(m, -1\,n + 1)\,9 + f\,(m, n - 1) + f(m, n) + f\,(m, n + 1) + f\,(m, +1n - 1) + f\,(m, +1n) + f\,(m, +1n + 1)\big)$$

Trade-off between noise removal and detail preserving:

Larger window -> can remove noise more effectively, but also blur the details/edges

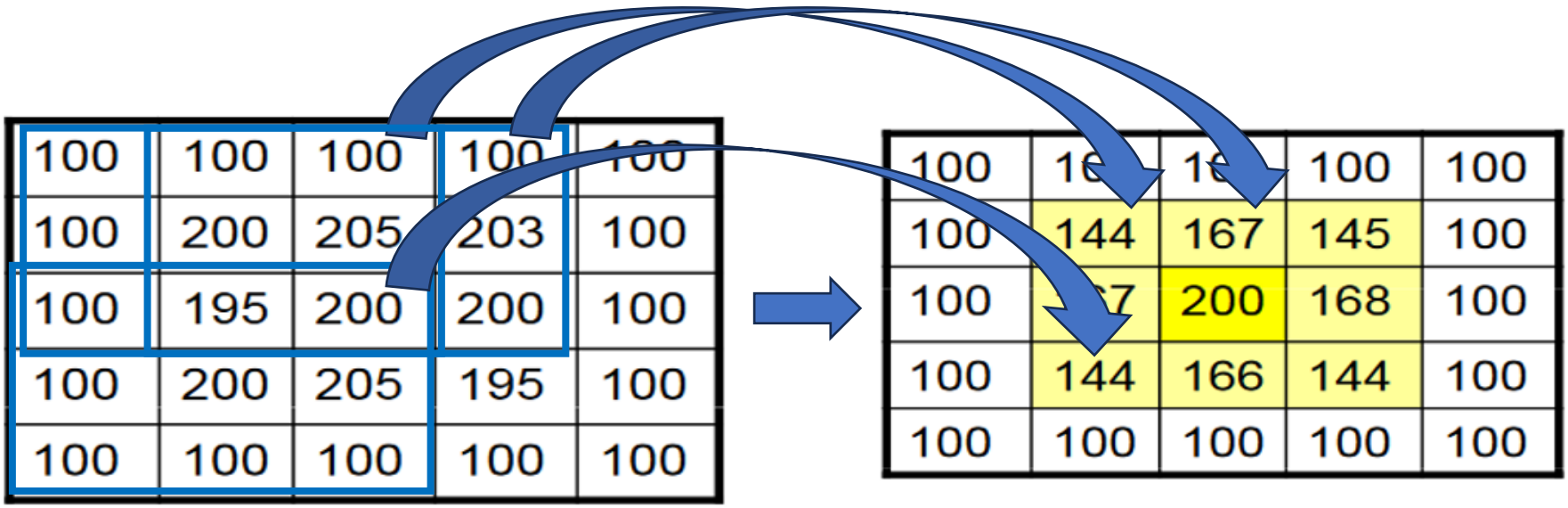
Example: 3x3 Averaging Linear Filters

$$g(2,2) = \frac{1}{9}(100 + 100 + 100 + 100 + \mathbf{200} + 205 + 100 + 195 + 200) = 144$$

$$g(2,3) = \frac{1}{9}(100 + 100 + 100 + 200 + \mathbf{205} + 203 + 195 + 200 + 200) = 167$$

$$g(2,4) = \frac{1}{9}(100 + 100 + 100 + 200 + 205 + \mathbf{203} + +100 + 195 + 200 + 200) = 145$$

$$\therefore g(3,2) = 167, g(3,3) = 200, g(3,4) = 168, g(4,2) = 144, g(4,3) = 166, g(4,4) = 144$$



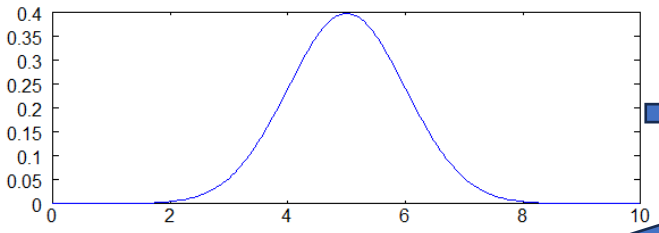
Examples: Weighting Smoothing Kernels/Mask

Examples of smoothing kernels:
(a) is a *box* kernel;
(b) is a *Gaussian* kernel.

$\frac{1}{9}\times$

1	1	1
1	1	1
1	1	1

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



0.075	0.124	0.075
0.124	0.204	0.124
0.075	0.124	0.075

$\frac{1}{16}\times$

1	2	1
2	4	2
1	2	1

1/16

1	2	1
2	4	2
1	2	1

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

1/1003

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0

All weights must sum to one

Examples: Weighting Smoothing Kernels/Mask

$$g(2,2) = \frac{1}{9}(1 \times 100 + 1 \times 100 + 1 \times 100 + 1 \times 100 + 1 \times 200 + 1 \times 205 + 1 \times 100 + 1 \times 195 + 1 \times 200) = 144$$

$$g(2,2) = \frac{1}{16}(1 \times 100 + 2 \times 100 + 1 \times 100 + 2 \times 100 + 4 \times 200 + 2 \times 205 + 1 \times 100 + 2 \times 195 + 1 \times 200) = 156$$

100	100	100	100	100
100	200	205	203	100
100	195	200	200	100
100	200	205	195	100
100	100	100	100	100

$\frac{1}{9}\times$

1	1	1
1	1	1
1	1	1

$\frac{1}{16}\times$

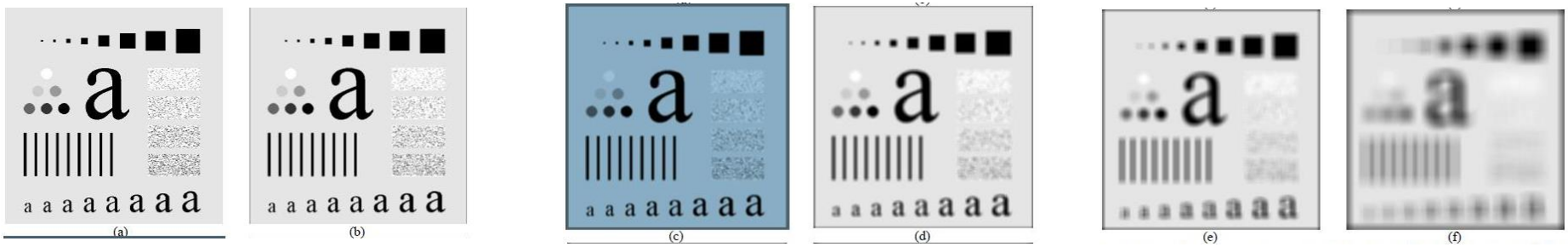
1	2	1
2	4	2
1	2	1

100	100	100	100	100
100	144	167	145	100
100	167	200	168	100
100	144	166	144	100
100	100	100	100	100

100	100	100	100	100
100	156	176	158	100
100	174	201	175	100
100	156	175	156	100
100	100	100	100	100

Effect of Averaging Filter

Effect of averaging filter. (a) Original image. (b)-(f) Results of smoothing with square averaging filter masks of sizes n = 3,5,9,15, and 35, respectively.



1. Blurring which is increased whenever the mask size increases.
2. Blending (removing) small objects with the background. The size of the mask establishes the relative size of the blended objects.
3. Black border because of padding the borders of the original image.
4. Reduced image quality.

Order Statistics Filter

are nonlinear spatial filters whose response is based on ordering (ranking) the pixels contained in the neighborhood, and then replacing the value of the center pixel with the value determined by the ranking result. It is included Max, Min, and Median filters.

Order Statistics Filter

- Ordering encompassed by the filter. Replaced center
- pixel with the value determined by the ranking result.
- Edges are better preserved in this filtering.

Types of Order statistics filter:

(i) Minimum filter: 0th percentile filter is the min filter. Center is replaced by the **smallest** value in the window.

(ii) Maximum filter: 100th percentile filter is the max filter. Center is replaced by the **largest** value in the window.

(iii) Median filter: Each pixel is considered. First neighboring pixels are sorted and original values of the pixel is replaced by the **median** of the list.

Median Filtering for Image Processing

Median filtering replaces each pixel value with the **median** (middle) value of its neighborhood. It's a **non-linear filter** — uses sorting, not averaging. Median filtering is a non-linear image filtering technique used primarily for noise reduction, especially effective against impulse noise (also known as *salt-and-pepper noise*). Unlike linear filters (such as averaging or Gaussian filters), which can blur edges, median filters preserve edges while removing noise.

Basic Idea

For each pixel in the image:

1. Consider a **neighborhood window** (e.g., 3×3, 5×5, etc.).
2. Sort the pixel intensity values within that window.
3. Replace the **center pixel** with the **median** (middle) value of the sorted list.

Median Filtering for Image Processing

Mathematical Expression

Let $f(x, y)$ be the input image, and $g(x, y)$ be the filtered output.

$$g(x, y) = median\{f(s, t) | (s, t) \in W(x, y)\}$$

where $W(x, y)$ is the neighborhood (e.g., a 3×3 window) centered at (x, y).

Common Applications

- Removing **salt-and-pepper noise**
- **Preprocessing** before segmentation or edge detection
- Cleaning **medical** or **radar** images
- Enhancing **binary/mask** images

Median Filtering for Image Processing

Effect

- Removes salt-and-pepper noise.
- Preserves edges better than mean filter.

Advantages

- Excellent for impulse noise.
- Preserves sharpness and edges.

Disadvantages

- Computationally heavier (sorting step).
- May distort small fine details if window too large.

Median Filtering for Image Processing

Comparison with Mean Filter

Feature	Mean Filter	Median Filter
Type	Linear	Non-linear
Operation	Average of neighbors	Median of neighbors
Removes	Gaussian noise	Salt-and-pepper noise
Edge Preservation	Poor	Excellent
Computation	Fast	Slower (sorting)
Effect	Smooths, blurs	Cleans, sharp edges
Window Type	Square (3×3, 5×5, etc.)	Square or cross-shaped

Median Filtering for Image Processing

A noisy 5×5 grayscale image with a salt-and-pepper noise pixel at position (3,3):

	1	2	3	4	5
1	50	52	54	53	51
2	49	51	53	52	50
3	50	49	200	51	52
4	48	50	52	50	49
5	47	49	51	50	48

Step 1: Define the filtering window

- Use a 3×3 window around the target pixel.
- Replace the center pixel by the median of the pixels in the window.

Median Filtering for Image Processing

Step 2: Apply median filter at pixel (3,3)
3×3 neighborhood around (3,3):

51	53	52
49	200	51
50	52	50

Step 3: Sort the neighborhood pixels
Values: 49,50,50,51,51,52,52,53,200
Sorted: 49,50,50,51,51,52,52,53,200
Median is the middle value (5th value): **51**

Median Filtering for Image Processing

Step 4: Replace pixel (3,3)

- Original pixel value = 200 (noise)
- New pixel value = 51 (median of neighbors)

Step 5: Result

The noise spike (200) is removed and replaced by a value consistent with neighbors, preserving image edges better than averaging.

Pixel Position	Original Value	New Value (Median Filtered)
(3,3)	200	51

Median Filtering for Image Processing

Step 4: Replace pixel (3,3)

- Original pixel value = 200 (noise)
- New pixel value = 51 (median of neighbors)

Step 5: Result

The noise spike (200) is removed and replaced by a value consistent with neighbors, preserving image edges better than averaging.

Pixel Position	Original Value	New Value (Median Filtered)
(3,3)	200	51

Median Filter

- Replace at center by median pixel value in the neighborhood, (i.e. middle element after they are sorted).
- Median filters are particularly useful in removing impulse noise (known as salt-and-pepper noise).
- Salt = 255, pepper = 0 gray levels.
- In a 3×3 neighborhood the median is the 5th largest value, in a 5×5 neighborhood the 13th largest value, and so on.
- e.g., let, 3×3 neighborhood has gray levels (10, 20, 0, 20, 255, 20, 20, 25, 15).

Sorted: (0,10,15,20,**20**,20,20,25,255),
median ⇒20, replaces the original pixel value 255 (salt noise).

Original image= 10, 20, 0, 20, 255, 20, 20, 25, 15
Filtered image= 0, 10, 15, 20, **20**, 20, 20, 25, 255

Median Filter

Example:

Consider the following 5×5 image:

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Apply a 3×3 median filter on the shaded pixels, and write the filtered image.

Solution

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Sort:
20, 25, 30, 30, **30**, 70, 80, 80, 255

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Sort
0, 20, 30, 70, **80**, 80, 100, 100, 255

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Sort
0, 70, 80, 80, **100**, 100, 110, 120, 130

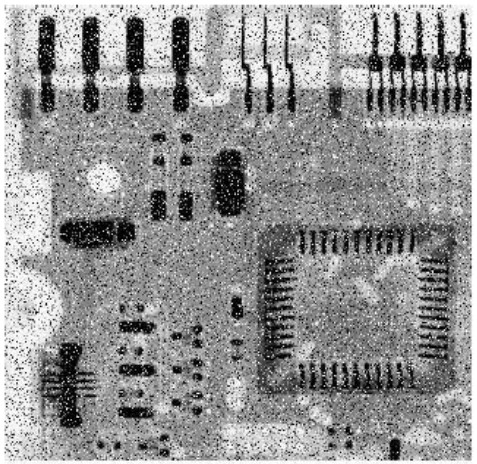
Filtered Image =

20	30	50	80	100
30	20	80	100	110
25	**30**	**80**	**100**	120
30	30	80	100	130
40	50	90	125	140

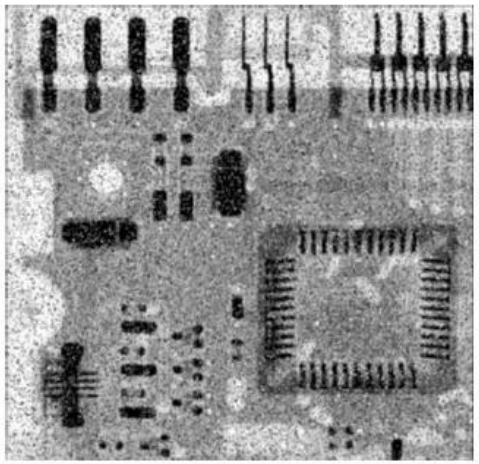
Effect of Median filter

(a) Image corrupted by salt & pepper noise. (b) Result of applying 3×3 standard averaging filter on (a). (c) Result of applying 3×3 median filter on (a). Effects of median filter are:

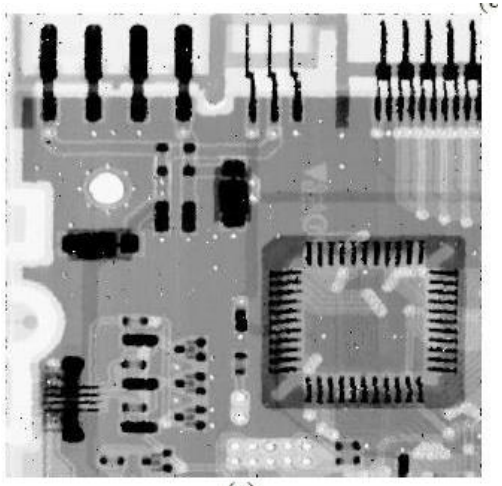
1. Noise reduction
2. Less blurring than averaging linear filter



(a)



(b)



(c)

Example: Using image addition (averaging) for noise reduction.

$$g(x, y) = f(x, y) + \eta(x, y)$$

Uncorrelated Corrupted image

Uncorrelated Noiseless image

Uncorrelated noise

(x, y) the noise is uncorrelated and has zero average value.

Example: Using image addition (averaging) for noise reduction.

$$\bar{g}(x,y) = \frac{1}{K}\sum_{i=1}^{K} g_i(x,y)$$

image averaging K different noisy images

Set of noisy image

$$\sigma_{\bar{g}(x,y)} = \frac{1}{\sqrt{K}}\sigma_{\eta(x,y)}$$

Expected value:

$$E\{\bar{g}(x,y)\} = f(x,y)$$

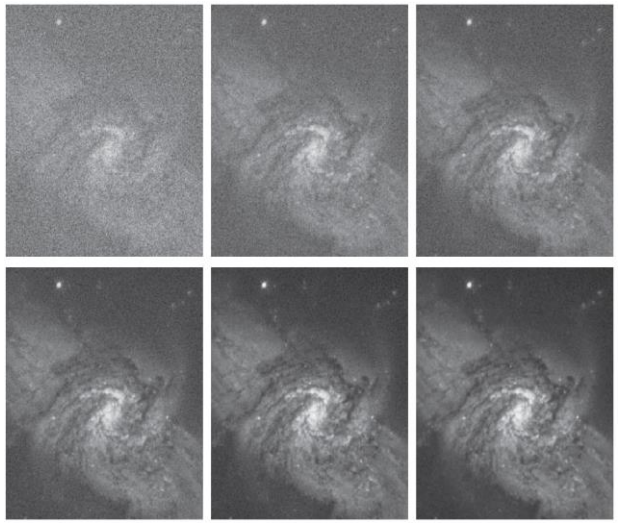
variance value:

$$\sigma^2_{\bar{g}(x,y)} = \frac{1}{K}\sigma^2_{\eta(x,y)}$$

S.D. value:

Example: Using image addition (averaging) for noise reduction.

As K increases, indicate that the variability (as measured by the variance or the standard deviation) of the pixel values at each location (x, y) decreases.



Because, $E\{g(x, y)\}=f(x,y)$; Thus, $g(x, y)$ approaches the noiseless image $f(x, y)$ as the number of noisy images used in the averaging process increases.

Reduce Noise by Averaging Neighboring Pixels

52	55	61	59	50
49	50	55	52	48
51	52	200	54	53
50	51	53	52	49
48	49	52	50	47

Filter pixel at position (3,3) (noisy pixel)

50	55	52
52	200	54
51	53	52

Average $New\ pixel\ value = \frac{619}{9} \approx 68.78 \approx 69$

52	55	61	59	50
49	50	55	52	48
51	52	69	54	53
50	51	53	52	49
48	49	52	50	47

- Original pixel (3,3) = 200 (very high, noise)
- After filtering = 69 (much closer to neighbors)

Separable Filter Kernels

$$G(x,y) = G_1(x)G_2(y)$$

2 × 3 Kernel

$$w = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \qquad \mathbf{c} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{r} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\mathbf{c}\,\mathbf{r}^T = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = w$$

Separable Kernel of size ***m*** ×n: $w = \mathbf{v}\mathbf{w}^T$

Separable square Kernel of size ***m*** ×m: $w = \mathbf{v}\mathbf{v}^T$

Separable Filter Kernels

Two simpler Kernel: $\boldsymbol{\omega = \omega_1 \times \omega_2}$

Follows the commutative and associative properties:

$$w \star f = (w_1 \star w_2) \star f = (w_2 \star w_1) \star f = w_2 \star (w_1 \star f) = (w_1 \star f) \star w_2$$

Computational advantages of performing convolution with separable as opposed to non-separable, kernel:

$$C = \frac{MNmn}{MN(m+n)} = \frac{mn}{m+n}$$

Thank You

Lecture 2.5

Spatial Filtering and Kernels for Smoothing Intensities, LPF, HPF, Comparisons between Filtering in Spatial and Frequency Domain

Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Low-Pass Filtering

$$\begin{bmatrix} 10 & 20 & 30 \\ 20 & \mathbf{40} & 20 \\ 30 & 20 & 10 \end{bmatrix} \qquad K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & \mathbf{1} & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Weighted neighbors =

$$= \frac{1}{9} \times (10 + 20 + 30 + 20 + \mathbf{40} + 20 + 30 + 20 + 10) = 22.22$$

Calculate new value at a corner pixel (0,0) using zero-padding

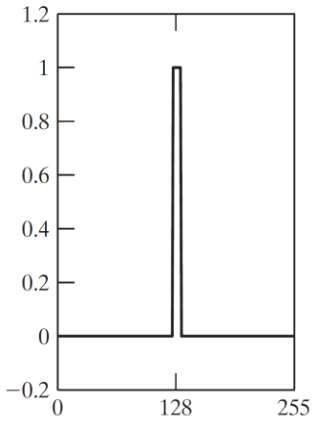
Neighbors (considering out-of-bound pixels as 0):

$$\frac{1}{9} \times (0 + 0 + 0 + 0 + 10 + 20 + 0 + 20 + 40) = \frac{1}{9} \times 90 = 10$$

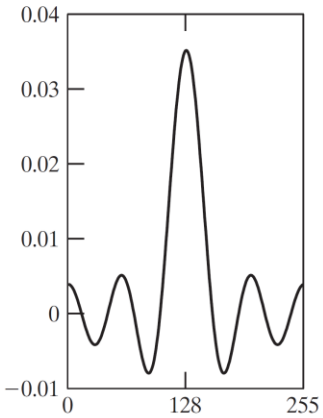
Smoothed Image (approximate)

$$I_{filtered} \approx \begin{bmatrix} 10 & \dots & \dots \\ \dots & \mathbf{22.22} & \dots \\ \dots & \dots & \dots \end{bmatrix}$$

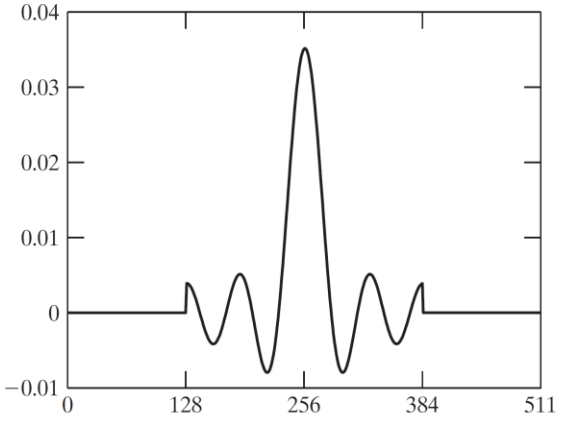
1-D Filtering



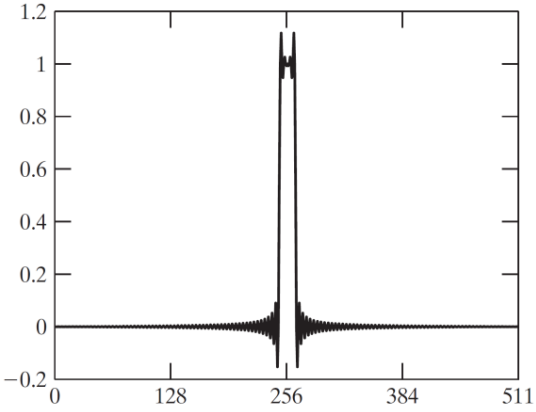
1-D ideal LPF in Frequency domain



IDFT for 1-D ideal LPF
Frequency multiplying by $(-1)^u$



Zero-padding



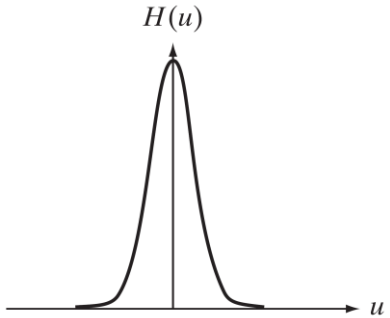
DFT padded filter

1-D Gaussian & Spatial LPF

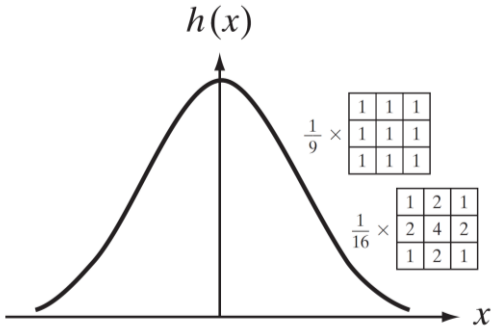
$$H(u) = Ae^{-u^2/2\sigma^2}$$

$$h(x) = \sqrt{2\pi}\sigma A e^{-2\pi^2\sigma^2x^2}$$

$$g(n) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{n^2}{2\sigma^2}}, n = -\infty \ldots\ldots \infty$$



1-D Gaussian LPF in Frequency domain



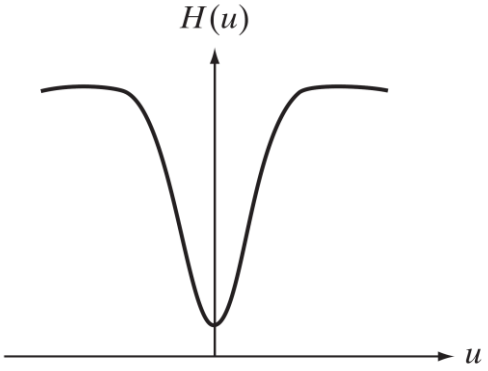
Spatial LPF in Frequency domain

1-D Gaussian & Spatial HPF

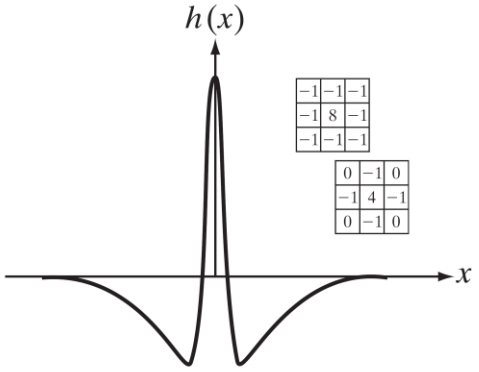
$$H(u) = Ae^{-u^2/2\sigma_1^2} - Be^{-u^2/2\sigma_2^2}$$

with $A \geq B$ and $\sigma_1 > \sigma_2$. The corresponding filter in the spatial domain is

$$h(x) = \sqrt{2\pi}\sigma_1 Ae^{-2\pi^2\sigma_1^2 x^2} - \sqrt{2\pi}\sigma_2 Be^{-2\pi^2\sigma_2^2 x^2}$$



1-D Gaussian HPF in Frequency domain



2-D mask Spatial filter

1-D Gaussian & Spatial LPF

$1 - D\ signal$: $f = [10, 20, 30, 40, 50]$

1-D Gaussian Kernel (size = 3, σ ≈ 1)

Common 3-point Gaussian kernel (normalized):

$$g = [1/4, 2/4, 1/4] = [0.25, 0.5, 0.25]$$

This is a **spatial low-pass filter**: it gives **more weight to center**, smooths noise.

Convolution (skipped edges):

$$0.25 \times 10 + 0.5 \times 20 + 0.25 \times 30 = 2.5 + 10 + 7.5 = 20.0$$
$$0.25 \times 20 + 0.5 \times 30 + 0.25 \times 40 = 5 + 15 + 10 = 30.0$$
$$0.25 \times 30 + 0.5 \times 40 + 0.25 \times 50 = 7.5 + 20 + 12.5 = 40.0$$

$$f_{smoothed} = [20.0, 30.0, 40.0]$$

Gaussian Kernel to Smooth (blur) Grayscale Image

10	20	30
40	**50**	60
70	80	90

Define a 3×3 Gaussian kernel:

Normalized Gaussian kernel with **σ ≈ 1.0**:

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

This kernel sums to 16 and gives more weight to the center and its close neighbors.

$$I'_{center(2,2)} = \frac{(1\times 10 + 2\times 20 + 1\times 30 + 2\times 40 + 4\times 50 + 2\times 60 + 1\times 70 + 2\times 80 + 1\times 90)}{16}$$

$$= 50$$

$$I'(1,1) = \frac{(1\times 0 + 2\times 0 + 1\times 0 + 2\times 0 + 4\times 10 + 2\times 20 + 1\times 0 + 2\times 40 + 1\times 50)}{16}$$

$$= 13.125 \approx 13$$

Gaussian Kernel to Smooth (blur) Grayscale Image

10	20	30
40	**50**	60
70	80	90

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$I'(1,2) = \frac{(2 \times 10 + 4 \times 20 + 2 \times 30 + 1 \times 40 + 2 \times 50 + 1 \times 60)}{16} \approx 23$$

$$I'(1,3) = \frac{(2 \times 20 + 4 \times 30 + 1 \times 50 + 2 \times 60)}{16} \approx 21$$

13	23	21
	50	

Low Pass Gaussian Filter Kernels

For circularly symmetric (Isotropic kernel), the Gaussian kernel:

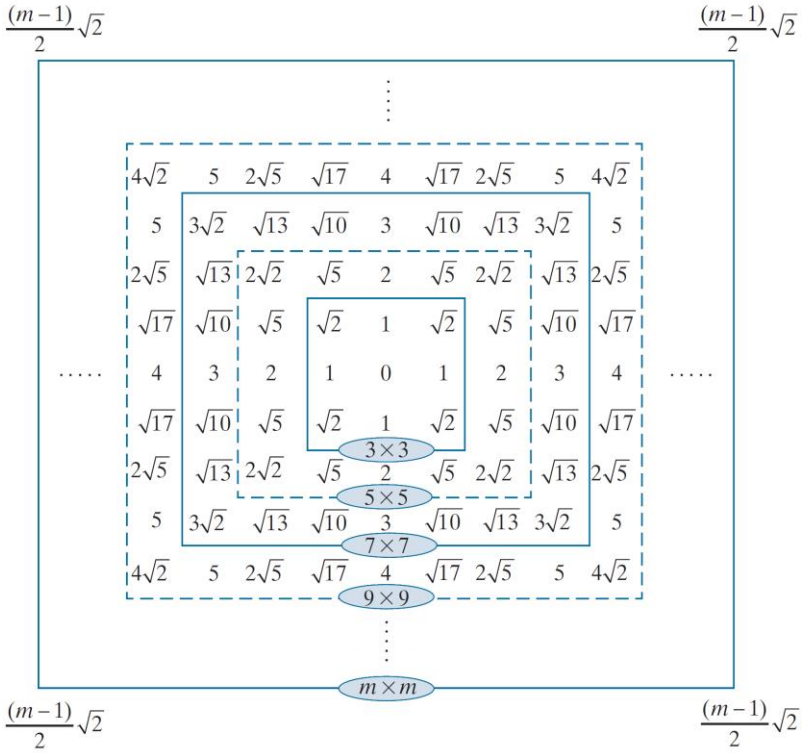
$$w(s,t) = G(s,t) = Ke^{-\frac{s^2+t^2}{2\sigma^2}}$$

r is distance from the center to any point on function G and for several kernel sizes using integer for s & t.

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}} \qquad r = \sqrt{s^2 + t^2}$$

Distance squared to corner points for kernel of size $\boldsymbol{m}$ ×m:

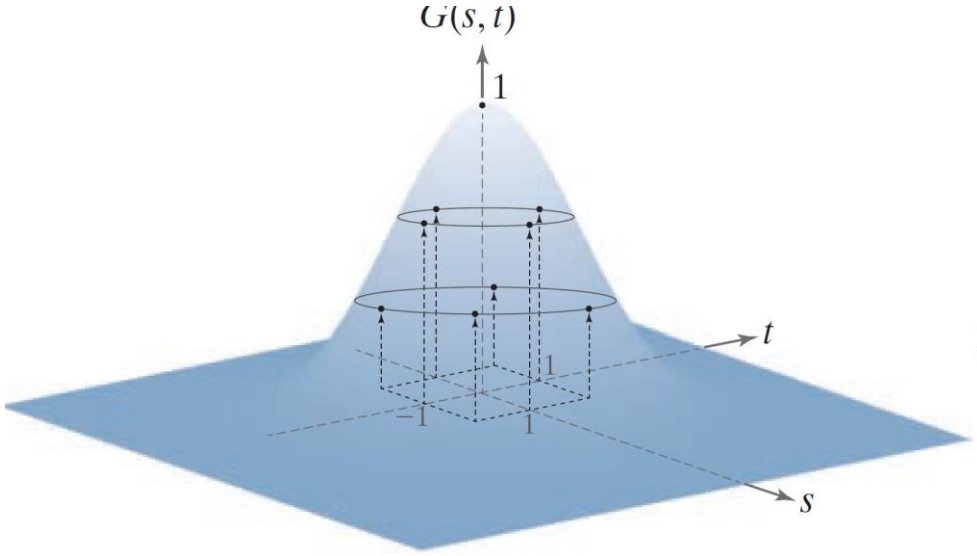
$$r_{\max}^2 = \left[\frac{(m-1)}{2}\sqrt{2}\right]^2 = \frac{(m-1)^2}{2}$$



Gaussian Kernel

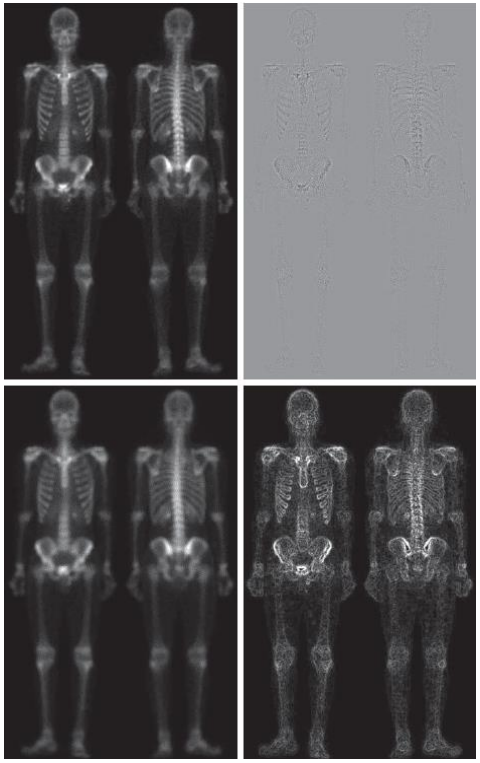
Sampling a Gaussian function to obtain discrete gaussian kernel.
For K=1, and $\sigma = 1, resulting$ 3× 3 kernel

	f	g	$f \times g$	$f \star g$
Mean	m_f	m_g	$m_{f\times g} = \dfrac{m_f\sigma_g^2 + m_g\sigma_f^2}{\sigma_f^2 + \sigma_g^2}$	$m_{f\star g} = m_f + m_g$
Standard deviation	σ_f	σ_g	$\sigma_{f\times g} = \sqrt{\dfrac{\sigma_f^2\sigma_g^2}{\sigma_f^2 + \sigma_g^2}}$	$\sigma_{f\star g} = \sqrt{\sigma_f^2 + \sigma_g^2}$



$$\frac{1}{4.8976} \times \begin{bmatrix} 0.3679 & 0.6065 & 0.3679 \\ 0.6065 & 1.0000 & 0.6065 \\ 0.3679 & 0.6065 & 0.3679 \end{bmatrix}$$

Gaussian Kernel



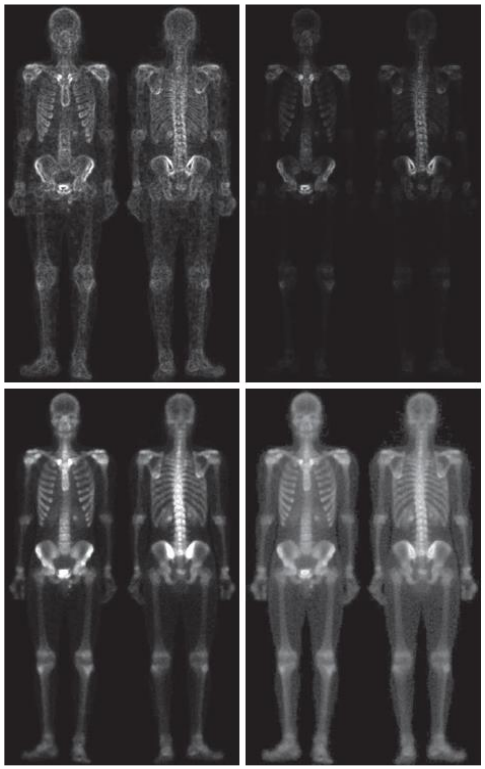
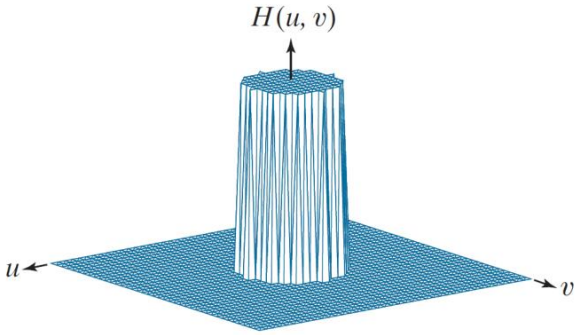


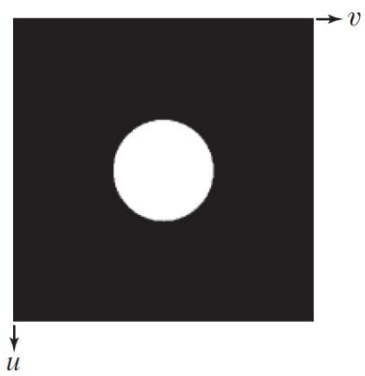
Image Smoothing Using Frequency Domain Filters: 2-D Ideal LPF (2-D ILPF)

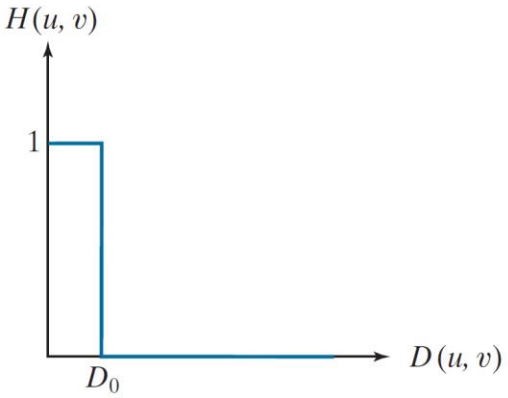
$$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \le D_0 \\ 0 & \text{if } D(u,v) > D_0 \end{cases}$$

$$D(u,v) = \left[(u - P/2)^2 + (v - Q/2)^2 \right]^{1/2}$$

P,Q: Padded sizes

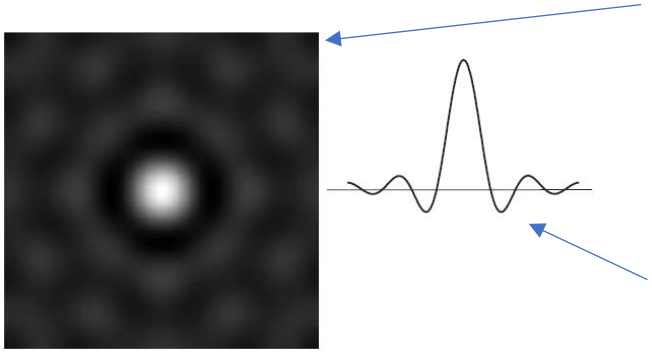






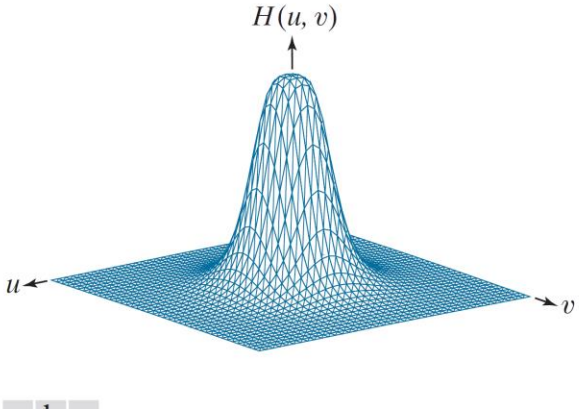
Butterworth LPF (BLPF)

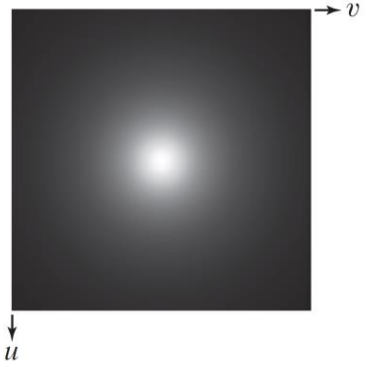
$$H(u,v) = \frac{1}{1 + \left[D(u,v)/D_0 \right]^{2n}}$$

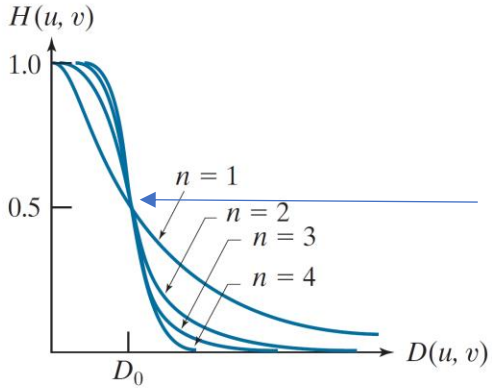


(a) Representation in the spatial domain of an ILPF of radius 5 and size 1000x1000

(b) Intensity profile of a horizontal line passing through the center of the image.



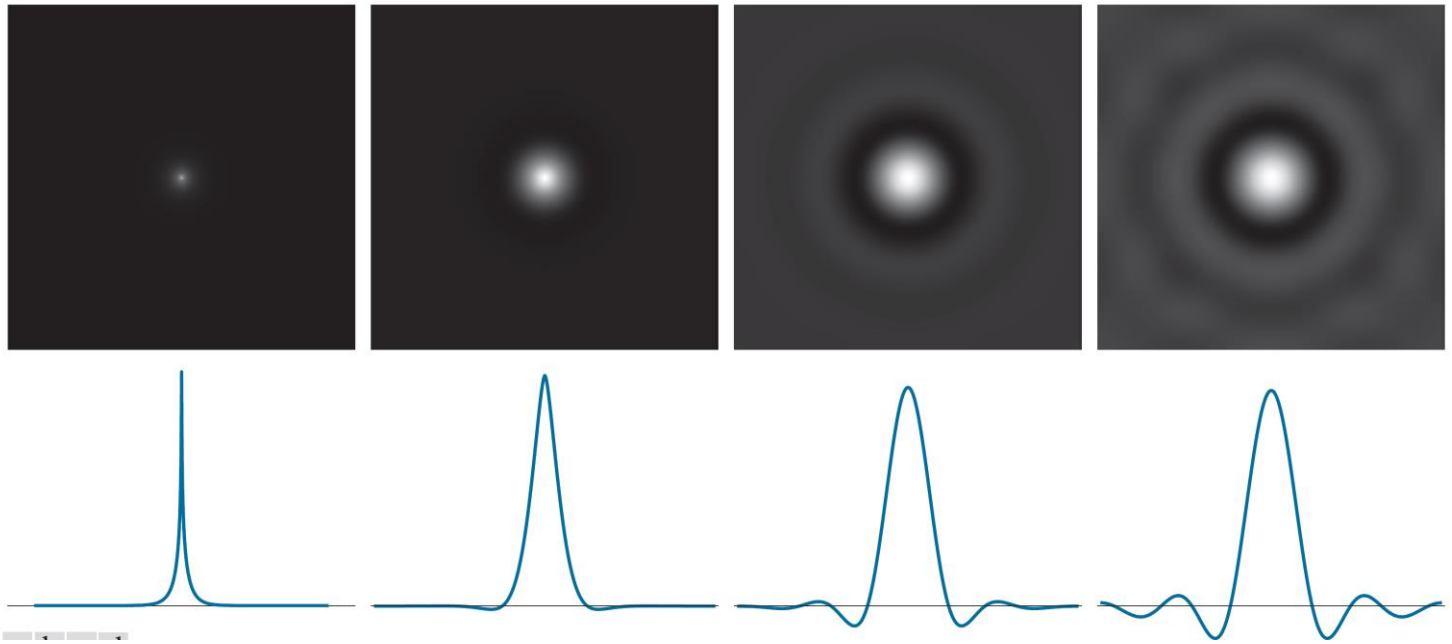




Down 50% from its maximum value of 1)

Butterworth LPF (BLPF)

Spatial representation of BLPF of order 1, 2, 5, and 20, and corresponding intensity profiles through the center of the filters (the size in all cases is and the cutoff frequency is 5). Observe how ringing increases as a function of filter order.



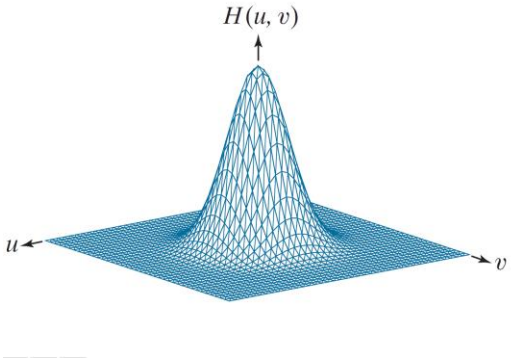
Gaussian LPF (GLPF)

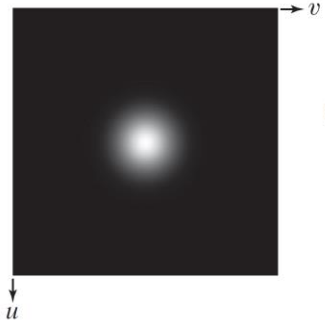
$$H(u,v) = e^{-D^2(u,v)/2\sigma^2}$$

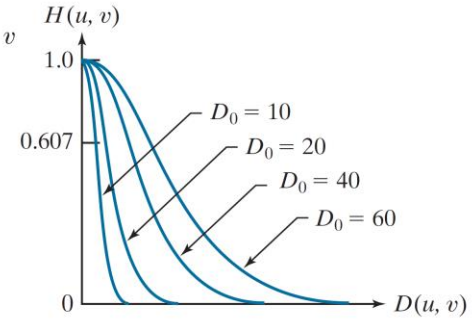
$$H(u,v) = e^{-D^2(u,v)/2D_0^2}$$

D(u,v): distance from the center of the frequency rectangle.

D(u,v)=D0







ILPF, BLPF & GLP

Ideal	Gaussian	Butterworth
$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \le D_0 \\ 0 & \text{if } D(u,v) > D_0 \end{cases}$	$H(u,v) = e^{-D^2(u,v)/2D_0^2}$	$H(u,v) = \dfrac{1}{1 + \left[D(u,v)/D_0\right]^{2n}}$



a b c

FIGURE 4.50 (a) Original image (784 × 732 pixels). (b) Result of filtering using a GLPF with $D_0 = 100$. (c) Result of filtering using a GLPF with $D_0 = 80$. Note the reduction in fine skin lines in the magnified sections in (b) and (c).

Filter Type	Transfer Function	Transition	Ringing	Smoothness	Common Use
ILPF	Step (cutoff at D_0)	Abrupt	High	Poor	Simple, theoretical
BLPF	$1/[1 + (D/D_0)^{2n}]$	Gradual (by n)	Moderate	Good	General smoothing
GLPF	$e^{-D^2/2D_0^2}$	Very smooth	None	Excellent	Natural blurring

2-D IHPF, 2-D BHPF, 2-D GHPF

A highpass filter is obtained from a given lowpass filter using the equation

$$H_{\mathrm{HP}}(u,v) = 1 - H_{\mathrm{LP}}(u,v)$$

Ideal	Gaussian	Butterworth
$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$	$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$	$H(u,v) = \dfrac{1}{1 + \left[D_0/D(u,v)\right]^{2n}}$

2-D IHPF, 2-D BHPF, 2-D GHPF

Ideal

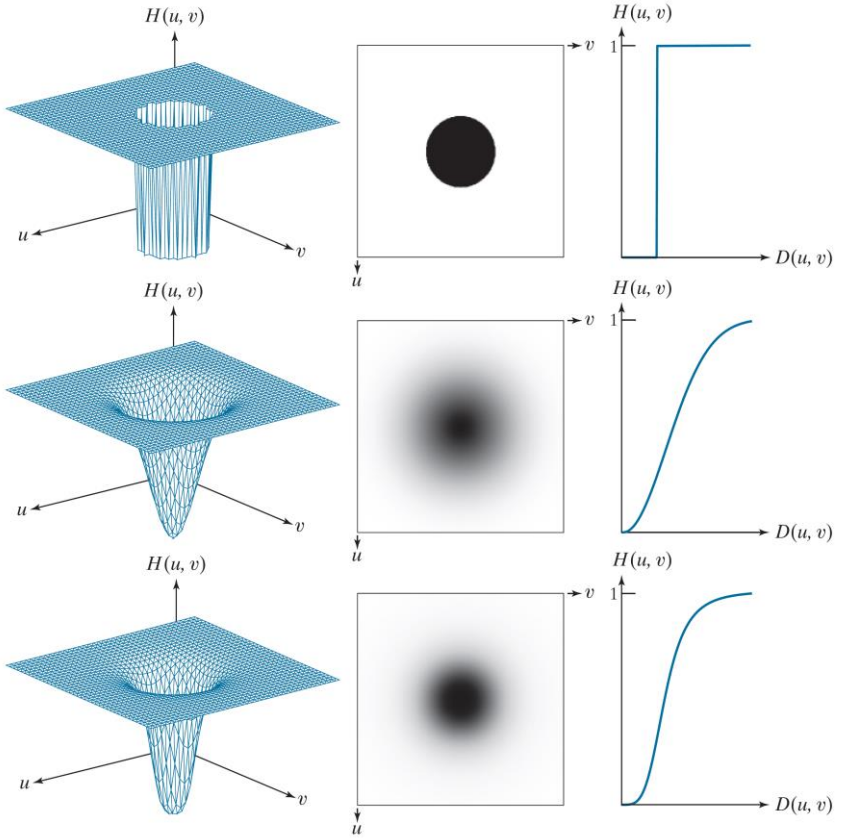
$$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$$

Gaussian

$$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$$

Butterworth

$$H(u,v) = \frac{1}{1 + \left[D_0/D(u,v)\right]^{2n}}$$



Finger print detection

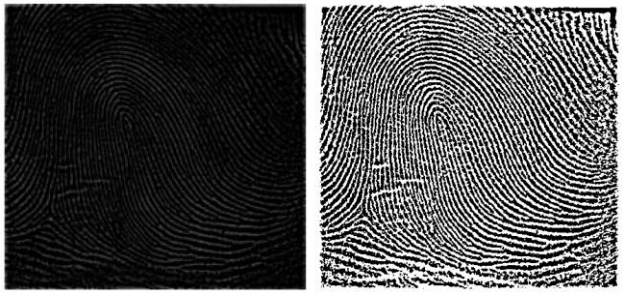
Finger print detection

Fingerprint Detection, also known as Fingerprint Image Processing and Recognition, a key application of biometric image analysis.

Fingerprint detection means identifying and analyzing the **ridge–valley patterns** in a fingerprint image to extract unique features (minutiae) for **authentication, identification, or matching**.

It is the **first stage** in the fingerprint recognition system — detecting, enhancing, and segmenting the fingerprint pattern.





Types of Fingerprint Patterns

Type	Description
Arch	Ridges enter from one side and exit the other.
Loop	Ridges enter and exit from the same side.
Whorl	Circular or spiral pattern around the core.

Important Fingerprint Features

These features form the **minutiae map**, unique to each person.

Type	Description
Ridge Ending	A ridge ends abruptly
Bifurcation	A ridge splits into two
Island / Dot	Small ridge segment
Enclosure	Ridge loop surrounds a valley

Major Steps in Finger print detection

Step	Process	Description
1. Image Acquisition	Capture	Fingerprint is obtained via scanner or sensor (optical, capacitive, or ultrasonic).
2. Preprocessing	Enhancement & noise reduction	Improve quality: grayscale conversion, normalization, and filtering.
3. Segmentation	ROI extraction	Separate fingerprint area from background.
4. Enhancement	Ridge structure improvement	Use filters (e.g., Gabor) or FFT to enhance ridges.
5. Binarization	Convert to binary (0/1)	Ridges → black (1), valleys → white (0).
6. Thinning	Skeletonization	Reduce ridges to 1-pixel width.
7. Minutiae Detection	Feature extraction	Detect ridge endings and bifurcations.
8. Matching	Comparison	Match extracted minutiae against database templates.

Techniques in Finger print detection

Purpose	Technique
Noise Reduction	Gaussian / Median Filter
Contrast Enhancement	Histogram Equalization
Edge Detection	Sobel or Canny operators
Ridge Orientation Estimation	Gradient calculation
Frequency Enhancement	Gabor or Fourier domain filtering
Minutiae Extraction	Crossing number or morphological operations

Finger print detection

- For finger print detection, it is stated that how you would justify the choice of cutoff frequency D_0 for a high-pass filtering step in fingerprint detection or enhancement.
- In fingerprint image processing, high-pass filters (such as Gaussian, Butterworth, or Ideal HPFs) are used to enhance ridge and valley details by suppressing low-frequency background variations (like uneven illumination, shading, or sensor noise).

Finger print detection

- For fingerprint detection, the cutoff frequency D_0=50 is chosen to be approximately 2.5% of the short dimension of the padded image. This ensures that the filter cutoff remains close to the origin, allowing low-frequency components (background and illumination variations) to be attenuated but not completely eliminated.
- In practice, selecting D_0 within 2%–5% of the image's short dimension provides a good balance between background suppression and ridge–valley detail preservation.
- The value $D_0 = 50$ is approximately 2.5% of the short dimension of the padded image. The idea is for D_0 to be close the origin so low frequencies are attenuated, but not completely eliminated. A range of 2%-5% of the short dimension is a good starting point.



Role of D_0=50

- D_0 is the cutoff frequency radius in the 2D frequency spectrum.
- In an image of size M×N, the **frequency domain** representation is usually padded (e.g., to 512×512 or 1024×1024) to reduce circular convolution artifacts.
- For example, if your padded fingerprint image is 2000×2000 pixels,
- 2.5%×2000=50.
- So, D_0=50 means we're filtering out frequencies near the center (the smooth background) but keeping the medium-to-high frequencies that contain ridge–valley patterns.



Why choose 2%–5% of the short dimension?

- The center of the frequency plane ($u = 0, v = 0$) represents low frequencies — smooth or slowly varying image content (illumination, background).
- If D_0 is *too small*, you remove too much low-frequency information → image looks overly sharp or noisy.
- If D_0 is *too large*, you barely filter out low frequencies → no visible enhancement.
- So, choosing $D_0 \approx$ 2%–5% of the image size gives a balanced result — enough to enhance edges and fine details without losing important structures.

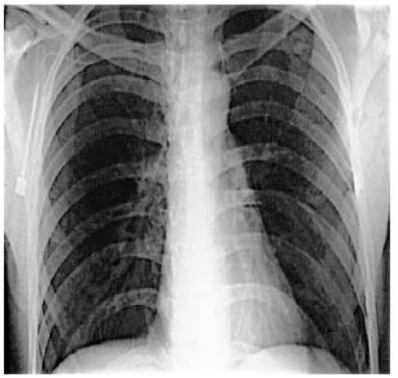
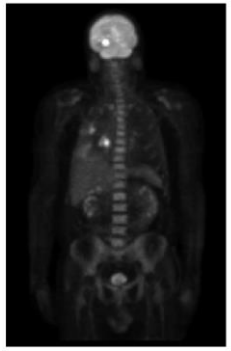
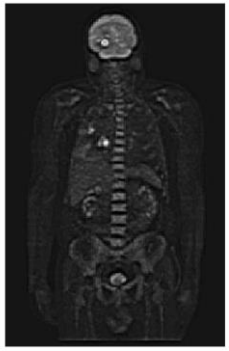


2-D IHPF, 2-D BHPF, 2-D GHPF

Typical Observations on X-ray Images

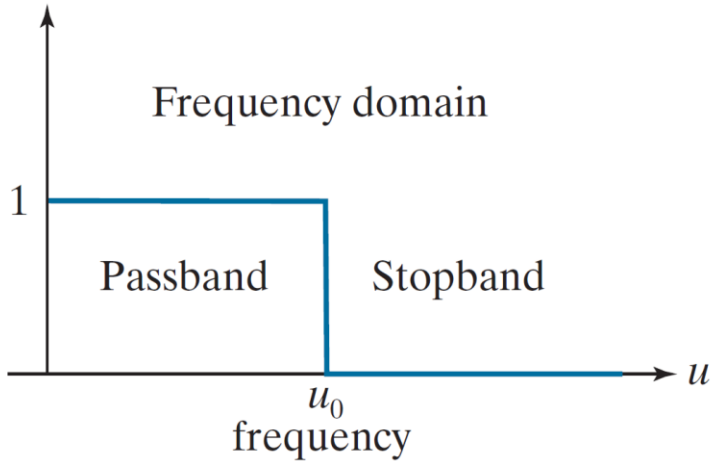
Filter	Effect
IHPF	Bones and fractures very sharp, but edges may have halos
BHPF	Edges enhanced, subtle fractures visible, minimal ringing
GHPF	Very smooth highlighting, best for subtle details and thin structures



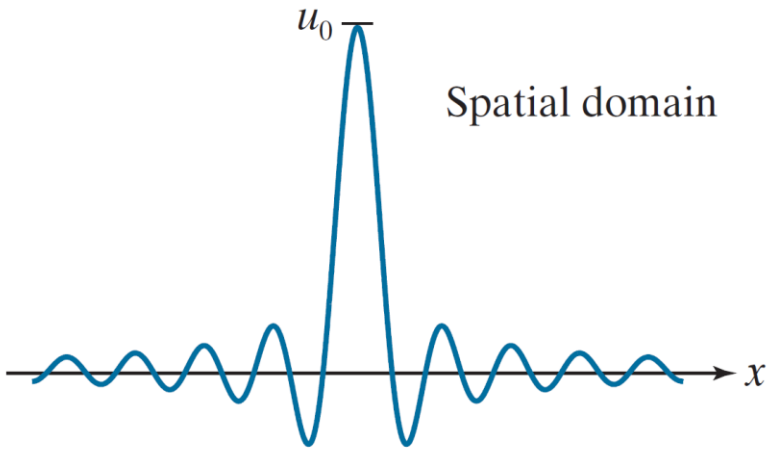





Comparisons between Filtering in Spatial and Frequency Domain

Ideal 1-D low-pass filter transfer function in the frequency domain (*Fourier transform*)



Corresponding filter kernel in the spatial domain (*Inverse Fourier transform*)



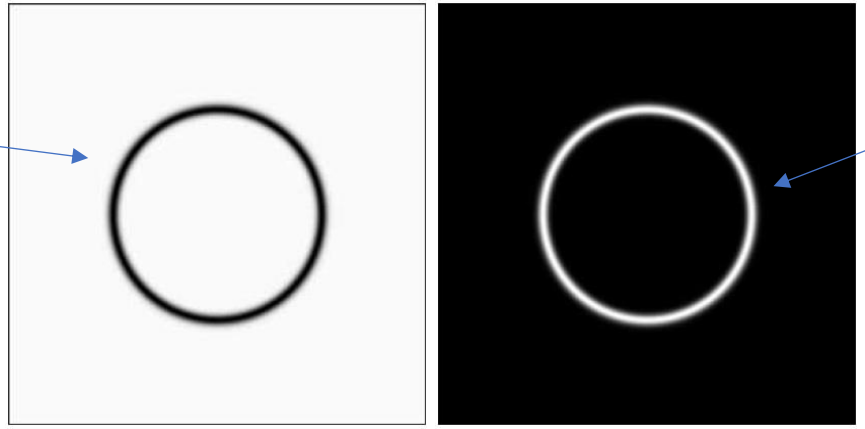
Bandreject and Bandpass Filters

$$H_{\mathrm{BP}}(u, v) = 1 - H_{\mathrm{BR}}(u, v)$$

Similar to HPF

0: Black
1: White

Gaussian bandreject filter



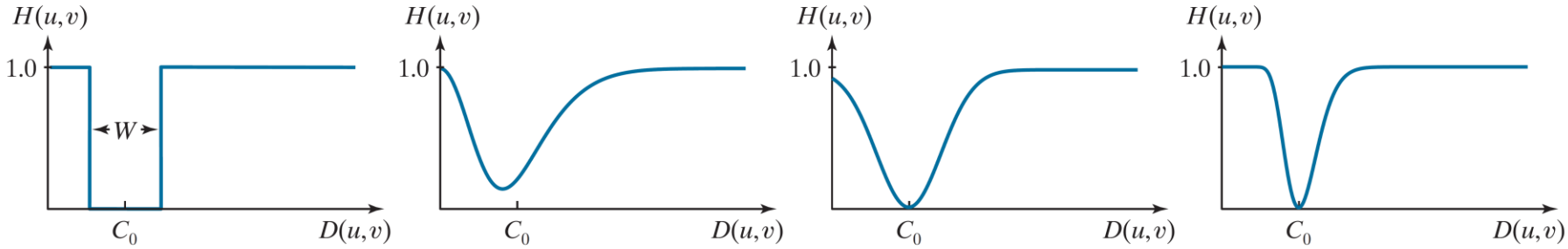
Gaussian bandpass filter

Bandreject and Bandpass Filters

TABLE 4.7
Bandreject filter transfer functions. C_0 is the center of the band, W is the width of the band, and $D(u,v)$ is the distance from the center of the transfer function to a point (u,v) in the frequency rectangle.

Ideal (IBRF)	Gaussian (GBRF)	Butterworth (BBRF)
$H(u,v) = \begin{cases} 0 & \text{if } C_0 - \frac{W}{2} \le D(u,v) \le C_0 + \frac{W}{2} \\ 1 & \text{otherwise} \end{cases}$	$H(u,v) = 1 - e^{-\left[\frac{D^2(u,v) - C_0^2}{D(u,v)W}\right]^2}$	$H(u,v) = \frac{1}{1 + \left[\frac{D(u,v)W}{D^2(u,v) - C_0^2}\right]^{2n}}$

Bandreject and Bandpass Filters

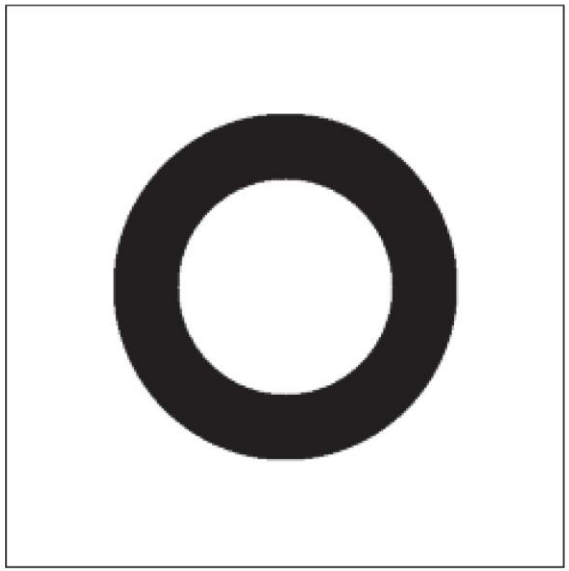


a b c d

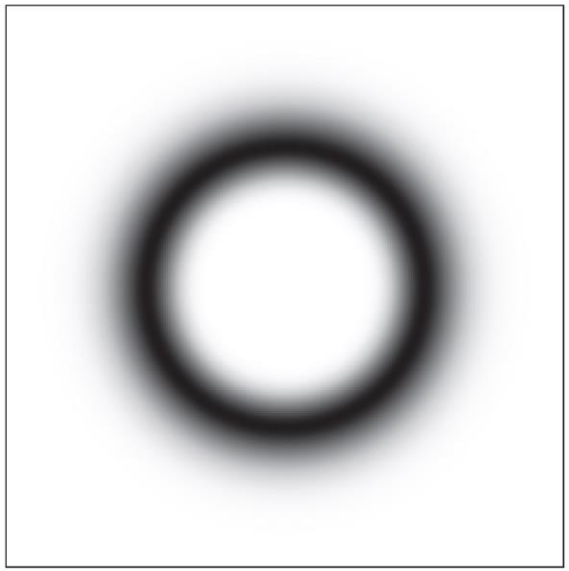
FIGURE 4.61 Radial cross sections. (a) Ideal bandreject filter transfer function. (b) Bandreject transfer function formed by the sum of Gaussian lowpass and highpass filter functions. (The minimum is not 0 and does not align with C_0.) (c) Radial plot of Eq. (4-149). (The minimum is 0 and is properly aligned with C_0, but the value at the origin is not 1.) (d) Radial plot of Eq. (4-150); this Gaussian-shape plot meets all the requirements of a bandreject filter transfer function.

Bandreject and Bandpass Filters

IBRF



GBRF



BBRF



Thank You

Lecture 2.6

Spatial Filtering and Kernels for Smoothing Intensities, Box Filter Kernels, The Laplacian Filter, Wavelet Transform and sub-band Decomposition, Image Enhancement in Frequency Domain

Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Box Filter Kernels

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix} \qquad K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Padding: $I_{pad} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 20 & 30 & 0 \\ 0 & 40 & 50 & 60 & 0 \\ 0 & 70 & 80 & 90 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

Calculate output pixel at position (1,1) (top-left corner in original):

$$I_{pad}(1,1) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 10 & 20 \\ 0 & 40 & 50 \end{bmatrix}$$

$$I'(1,1) = \frac{1}{9}\sum I_{pad}(1,1) = 10 + 20 + 0 + 40 + 50 = \frac{120}{9} = 13.33$$

Box Filter Kernels

Padding: $I_{pad} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 20 & 30 & 0 \\ 0 & 40 & 50 & 60 & 0 \\ 0 & 70 & 80 & 90 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

Calculate output pixel at position (1,2) (center pixel): $I_{pad}(1,2) = \begin{bmatrix} 0 & 0 & 0 \\ 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$

$$I'(1,2) = \frac{1}{9}\sum I_{pad}(1,1) = \frac{1}{9}(10 + 20 + 30 + 40 + 50) = \frac{210}{9}$$

$$= 23.33$$

Box Filter Kernels

Padding: $I_{pad} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 20 & 30 & 0 \\ 0 & 40 & 50 & 60 & 0 \\ 0 & 70 & 80 & 90 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

Calculate output pixel at position (2,2) (center pixel):

$$I_{pad}(2,2) = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$$

$$I'(2,2) = \frac{1}{9}\sum I_{pad}(1,1) = \frac{450}{9} = 50$$

$$I' = \begin{bmatrix} 13.33 & 23.33 & 16.67 \\ 23.33 & 50.00 & 36.67 \\ 16.67 & 36.67 & 26.67 \end{bmatrix}$$

Summary:

- The **box filter smooths the image** by averaging pixel values in a 3×3 neighborhood.
- Edge pixels are affected by zero-padding.
- Result is a blurred/smoothed image.

Box Filter Kernels

Padding: $I_{pad} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 20 & 30 & 0 \\ 0 & 40 & 50 & 60 & 0 \\ 0 & 70 & 80 & 90 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

Calculate output pixel at position (2,2) (center pixel):

$$I_{pad}(2,2) = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$$

$$I'(2,2) = \frac{1}{9}\sum I_{pad}(1,1) = \frac{450}{9} = 50$$

$$I' = \begin{bmatrix} 13.33 & 23.33 & 16.67 \\ 23.33 & 50.00 & 36.67 \\ 16.67 & 36.67 & 26.67 \end{bmatrix}$$

Summary:

- The **box filter smooths the image** by averaging pixel values in a 3×3 neighborhood.
- Edge pixels are affected by zero-padding.
- Result is a blurred/smoothed image.

The Laplacian Filter

The Laplacian operator of an image $f(x,y)$ is:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

This equation can be implemented using the 3×3 mask:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Since the Laplacian filter is a linear spatial filter, we can apply it using the same mechanism of the convolution process. This will produce a laplacian image that has grayish edge lines and other discontinuities, all superimposed on a dark, featureless background.

The Laplacian Filter

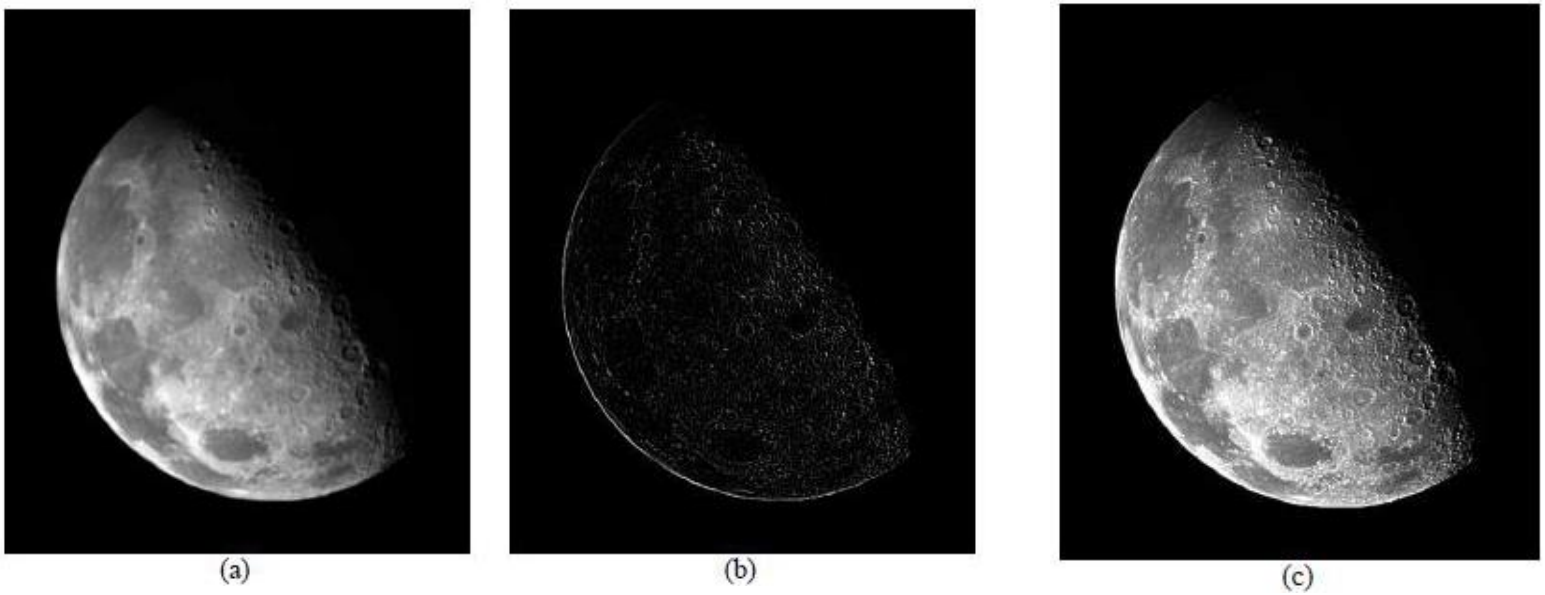
Background features can be "recovered" while still preserving the sharpening effect of the Laplacian operation simply by adding the original and Laplacian images.

$$g(x,y) = f(x,y) + \nabla^2 f(x,y)$$

The figure below shows an example of using Laplacian filter to sharpen an image.

Example of Applying Laplacian Filter

Example of applying Laplacian filter. (a) Original image. (b) Laplacian image. (c) Sharpened image.



(a) (b) (c)

Image Sharpening Filters

50	52	54
48	**50**	52
46	48	50

$$Laplacian\ kernel = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

52	54	0
50	**52**	0
48	50	0

$$(0 \times 50) + (-1 \times 52) + (0 \times 54) + (-1 \times 48) + (4 \times 50) + (-1 \times 52) + (0 \times 46) + (-1 \times 48) + (0 \times 50) = 0$$

$$(0 \times 52) + (-1 \times 54) + (0 \times 0) + (-1 \times 50) + (4 \times 52) + (-1 \times 0) + (0 \times 48) + (-1 \times 50) + (0 \times 0) = 54$$

Sharpened pixel value:

$$Original + Filtered = 50 + 0 = 50$$

$$Original + Filtered = 52 + 54 = 106$$

50	52	54
48	**50**	**106**
46	48	50

Interpretation:

- The center pixel value stayed the same (flat region).
- The pixel near an edge or detail (2,3) increased in intensity (from 52 to 106), highlighting edges.

Unsharp Masking Image Processing

50	52	54
48	**50**	**52**
46	48	50

Blurred pixel value = (50 + 52 + 54 + 48 + 50 + 52 + 46 + 48 + 50)/9=450/9=50

Assume the blurred pixel at (2,2) is 50.

At (2,3): Blurred =(52 + 54 + 0 + 50 + 52 + 0 + 48 + 50 + 0)/9= 306 / 9 = 34

At (2,2): Mask = Original – Blurred=50−50=0

$$Sharpened\ pixel = Original + k \times Mask$$
$$= 50 + 1 \times 0 = 50$$

At (2,3): Mask = Original – Blurred=52−34=18

$$Sharpened\ pixel = Original + k \times Mask$$
$$= 52 + 1 \times 18 = 70$$

Unsharp Masking Image Processing

50	52	54
48	**50**	**52**
46	48	50



50	52	54
48	**50**	**34**
46	48	50

Blurred Image



50	52	54
48	**50**	**70**
46	48	50

Sharpened Image

Summary Table:

Position	Original	Blurred	Mask (Orig - Blur)	Sharpened (Orig + Mask)
(2,2)	50	50	0	50
(2,3)	52	34	18	70

Interpretation:

- The blurred image removes fine details.
- The mask highlights edges and details.
- Adding the mask back sharpens the image by boosting edges.

Wavelet Transform and sub-band Decomposition

The **Wavelet Transform (WT)** analyzes an image at **multiple scales (resolutions)** using small localized waves called **wavelets**.

Common Wavelets Used

- Image Compression (JPEG2000)
- Image Denoising
- Edge Detection
- Texture Analysis & Feature Extraction
- Haar (simple, fast, exam favorite)
- Daubechies (db4, db8) – smooth, popular

Wavelet vs DCT (Quick Comparison)

Feature	DCT	Wavelet
Localization	Poor	Excellent
Blocking artifacts	Yes	No
Multi-resolution	No	Yes
Used in	JPEG	JPEG2000

Wavelet Transform and sub-band Decomposition

Example: 1D Signal, $x = [4,6,10,12]$

Haar Wavelet Filters

- **Low-pass filter (approximation):** $h = [1/\sqrt{2}, 1/\sqrt{2}]$
- **High-pass filter (detail):** $g = [\frac{1}{\sqrt{2}}, -1/\sqrt{2}]$

$$\psi(t) = \begin{cases} 1 & 0 \le t < \frac{1}{2}, \\ -1 & \frac{1}{2} \le t < 1, \\ 0 & \text{otherwise.} \end{cases}$$

scaling function $\varphi(t)$ can be described as

$$\varphi(t) = \begin{cases} 1 & 0 \le t < 1, \\ 0 & \text{otherwise.} \end{cases}$$

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$H_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix},$$

$$H_4 = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ \sqrt{2} & -\sqrt{2} & 0 & 0 \\ 0 & 0 & \sqrt{2} & -\sqrt{2} \end{bmatrix}$$

Wavelet Transform and sub-band Decomposition

Example: 1D Signal, $x = [4,6,10,12]$

Haar Wavelet Filters

- **Low-pass filter (approximation):** $h = [1/\sqrt{2}, 1/\sqrt{2}]$
- **High-pass filter (detail):** $g = [\frac{1}{\sqrt{2}}, -1/\sqrt{2}]$

Pair 1: Elements 4 and 6:

$$a1 = \frac{1}{\sqrt{2}} \times 4 + \frac{1}{\sqrt{2}} \times 6 \approx 7.07$$

$$d1 = \frac{1}{\sqrt{2}} \times 4 - \frac{1}{\sqrt{2}} \times 6 \approx -1.41$$

Pair 2: Elements 10 and 12:

$$a2 = \frac{1}{\sqrt{2}} \times 10 + \frac{1}{\sqrt{2}} \times 12 \approx 15.56$$

$$d2 = \frac{1}{\sqrt{2}} \times 10 - \frac{1}{\sqrt{2}} \times 12 \approx -1.14$$

Resulting Sub-bands:

Approx. coefficients (low-frequency):

$$A = [7.07, 15.56]$$

$$D = [-1.41, -1.41]$$

Detail coefficients (high-frequency):

Wavelet Transform and sub-band Decomposition

$A = [7.07, 15.56]$ $D = [-1.41, -1.41]$

One More Level of Decomposition on Approximation A

$$a = \frac{7.07 + 15.56}{1.414} \approx 16.0$$

$$d = \frac{7.07 - 15.56}{1.414} \approx -6.0$$

Summary:

Signal Component	Values
Original Signal	[4, 6, 10, 12]
Level 1 Approx.	[7.07, 15.56]
Level 1 Detail	$[-1.41, -1.41]$
Level 2 Approx.	[16.0]
Level 2 Detail	$[-6.0]$

Image Enhancement in Frequency Domain

Objective:
Enhance an image by **amplifying its high-frequency components** to sharpen edges.

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{20} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

The center pixel is brighter — an edge-like feature.

Compute the 2D Discrete Fourier Transform (DFT)
We compute DFT $F(u,v)F(u,v)$ for each frequency $(u,v)(u,v)$:

$$F(u,v) = \sum_{x=0}^{2}\sum_{y=0}^{2} I(x,y)e^{-j2\pi\left(\frac{ux}{3}+\frac{vy}{3}\right)}$$

Calculate a few important frequency components.

Image Enhancement in Frequency Domain

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{20} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

$$F(u,v) = \sum_{x=0}^{2} \sum_{y=0}^{2} I(x,y) e^{-j2\pi\left(\frac{ux}{3}+\frac{vy}{3}\right)}$$

DC Component F(0,0): Sum of all pixels:

$$F(0,0) = 10 + 10 + 10 + 10 + \mathbf{20} + 10 + 10 + 10 + 10 = 100$$

Image Enhancement in Frequency Domain

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{20} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

High-frequency Component $F(1,1)$:

$$F(1,1) = \sum_{x=0}^{2}\sum_{y=0}^{2} I(x,y)e^{-j2\pi\left(\frac{x}{3}+\frac{y}{3}\right)}$$

$$= 10 \times \left(e^{0} + e^{-\frac{j2\pi}{3}} + e^{-\frac{j4\pi}{3}}\right) + 10 \times \left(e^{-\frac{j2\pi}{3}} + e^{-\frac{j4\pi}{3}} + e^{-j2\pi}\right) + 10$$

$$\times \left(e^{-\frac{j4\pi}{3}} + e^{-j2\pi} + e^{-\frac{j8\pi}{3}}\right) = 10 \times \left(e^{0} + 2e^{-\frac{j2\pi}{3}} + 3e^{-\frac{j4\pi}{3}} + 2e^{-j2\pi} + e^{-\frac{j8\pi}{3}}\right)$$

$$= 10$$

$$\times \left[1 + 2\left(Cos\frac{2\pi}{3} - jsin\frac{2\pi}{3}\right) + 3\left(Cos\frac{4\pi}{3} - jsin\frac{4\pi}{3}\right) + 2(Cos\,2\pi - jsin\,2\pi)\right.$$

$$\left. + \left(Cos\frac{8\pi}{3} - jsin\frac{8\pi}{3}\right)\right]$$

$$= 10 \times \left[1 + 2\left(-\frac{1}{2} + j\frac{\sqrt{3}}{2}\right) + 3\left(-\frac{1}{2} + j\frac{\sqrt{3}}{2}\right) + 2(1 - j \times 0) + \left(-\frac{1}{2} - j\frac{\sqrt{3}}{2}\right)\right]$$

$$= j20\sqrt{3}$$

Image Enhancement in Frequency Domain

The calculation is a bit tedious, so let's simplify by noting:

- The high-frequency components correspond to edges and rapid intensity changes.
- Here, since center pixel is brighter, these components will have nonzero values.

Apply High-pass Filter in Frequency Domain

To enhance edges, multiply the frequency spectrum by a high-pass filter $H(u, v)$ that:

- Suppresses low frequencies (like DC)
- Amplifies high frequencies

A simple high-pass filter:

$$H(u,v) = \begin{cases} 0 & if\ (u,v) = (0,0) \\ 1 & otherwise \end{cases}$$

Image Enhancement in Frequency Domain

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{20} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Pixel	Original Intensity	Enhanced Intensity (Approx.)
Center (1,1)	20	Higher (sharper)
Neighbors	10	Lower or more contrasted

Wavelet Transform in Image Processing

	1	2	3	4
1	52	55	61	59
2	62	59	55	90
3	63	65	66	113
4	58	70	70	120

Apply 1D Haar wavelet transform on rows:

Approx. (A): average of pair $= \frac{x1+x2}{2}$

Detail (D): difference of pair $= \frac{x1-x2}{2}$

Pair 1		Pair 2	
A	D	A	D
(52+55)/2=53.5	(52-55)/2=-1.5	(61+59)/2=60	(61-59)/2=1
(62+59)/2=60.5	(62-59)/2=1.5	(55+90)/2=72.5	(55-90)/2=-17.5
(63+65)/2=64	(63-65)/2=-1	(66+113)/2=89.5	(66-113)/2=-23.5
(58+70)/2=64	(58-70)/2=-6	(70+120)/2=95	(70-120)/2=-25

Wavelet Transform in Image Processing

	1	2	3	4
1	52	55	61	59
2	62	59	55	90
3	63	65	66	113
4	58	70	70	120



A	A
53.5	60
60.5	72.5
64	89.5
64	95

D	D
-1.5	1
1.5	-17.5
-1	-23.5
-6	-25

A	A	D	D
(53.5+60.5)/2=57	(60+72.5)/2=66.25	(53.5-60.5)/2=-3.5	(60-72.5)/2=-6.25
(64+64)/2=64	(89.5+95)/2=92.25	(64-64)/2=0	(89.5-95)/2=-2.75

Final Wavelet Coefficients

57	**66.25**	**-3.5**	**-6.25**
64	**92.25**	**0**	**-2.75**
-1.5	**1**	**1.5**	**-17.5**
-1	**-23.5**	**-6**	**-25**

Thank You